

YAMBA — Cahier de résultats, recette API (cahier n° 3)

Yamba — documentation générée le 09/09/2026 depuis `context/YAMBA-RECETTE-API-RESULTATS.md`

Table des matières

En-tête de campagne

Chapitre 4 — Scénarios transverses

Anomalies du chapitre 4

Écarts du cahier (à corriger dans le cahier, pas dans le code)

Commentaire de chapitre

Suivi des anomalies

Chapitre 5.1 — auth-service (terminé)

Anomalies du chapitre 5.1

Écarts du cahier (chapitre 5.1)

Challenge expert — auth-service (au-delà des anomalies de recette)

1. isAuthenticated charge tout le document membre, à chaque requête, sur les six services
2. Aucune protection anti-force brute par compte sur /auth/login
3. La journalisation d'auth-service est en retard sur le reste de la plateforme
4. Deux fichiers de 900 lignes portent l'essentiel du service
5. Les rôles vivent dans le jeton, donc ils ont jusqu'à 15 minutes de retard
6. Deux détails à faible coût

Challenge expert — l'argent du membre : compte Stripe du Voyageur et finances du tableau de bord

- A — Le parcours Stripe du Voyageur : quatre défauts qui s'enchaînent
- B — Le bouton « tableau de bord Stripe » ne fonctionne pour personne
- C — Ce que le Voyageur voit de son argent, et ce qui lui manque
- D — Côté Expéditeur
- E — Code mort à retirer au passage
- Priorisation proposée
- F — L'onboarding est-il trop long ? Ce qu'on peut retirer sans rien perdre

Chapitre 5.2 — trip-service (17 fiches jouées sur 18)

Anomalies du chapitre 5.2

Écarts du cahier (chapitre 5.2)

Chapitre 5.3 — deal-service, le cœur transactionnel (18 fiches jouées sur 23)

Anomalies du chapitre 5.3

À arbitrer (le code contredit le cahier et s'en explique)

Chapitre 5.4 — message-service (11 fiches sur 12)

Une remarque transverse, mineure

Chapitre 5.5 — notification-service (4 fiches)

Anomalies du chapitre 5.5

Chapitre 6 — Sécurité (12 fiches jouées sur 15)

Anomalie du chapitre 6

Ce que ce chapitre m'a appris sur mes propres tests

Chapitre 7 — Idempotence et concurrence (10 fiches sur 10)

Anomalies du chapitre 7

La leçon transverse du chapitre

Chapitre 8 — Webhooks (8 fiches sur 8)

La preuve qui compte le plus, et comment elle a été obtenue

Comment les fiches Stripe ont été jouées sans la CLI Stripe

Rejeu avec la CLI Stripe — la réserve n° 1 est levée

Axe non demandé — les deux webhooks n'avaient aucun test de contrôleur

9.1 Ce que la campagne a couvert

9.2 Le tableau de suivi a lui-même été réparé

9.3 Les quatre critères de sortie, un par un

1. Zéro anomalie bloquante ouverte — TENU
2. Toutes les fiches bloquantes jouées et OK — TENU
3. Les majeures arbitrées une par une — TENU
4. Les mineures inscrites au journal de dette — TENU (ci-dessous)

9.4 Journal de dette au 8 septembre 2026

9.5 Ce que cette campagne NE prouve pas

9.6 Verdict

9.7 Ce que la campagne a appris, au-delà des anomalies

Ce qu'il y avait à faire

Deux défauts de sémantique sont tombés avec la dette

Les middlewares partagés — la dette D-5, à moitié soldée

Les garde-fous

Contre-épreuves

Un piège Nx, payé une deuxième fois

Ce qui a été fait

Deux choses que cette anomalie apprend

Et pourquoi la campagne ne l'avait pas vue

Le remède complet, et sa limite

Ce que le journal disait, et ce qu'il fallait lire

Le défaut

La correction

Trois statuts faux corrigés dans la même passe

Garde-fou

Contre-épreuves

L'arbitrage n'était pas à rendre : il était déjà au registre

La vraie dette était ailleurs, et le code la portait

La règle, écrite une fois

Garde-fous

Contre-épreuves sur le service

Document de campagne, rempli au fil des chapitres. Il complète le tableau de suivi du §9.1 de docs/recette/RECETTE-03-API.md : celui-ci porte le verdict, celui-là porte **le pourquoi, la recommandation et le commentaire**. Les anomalies sont numérotées AN0-API-<nn> (format §9.2).

En-tête de campagne

Champ	Valeur
Date	8 septembre 2026
Testeur	Gomab (assisté)
Branche / version	dev — 99bc1fc (après PR #230)
Base de données	MongoDB Atlas, base development (distante)
Fournisseur de paiement	FAKE — deal-service lancé depuis son bundle, STRIPE_SECRET_KEY vidée (§2.9). Preuve : POST /deals/payment-intents → {"provider":"FAKE","clientSecret":null}
Broker d'événements	actif — Redpanda yamba-redpanda, topics booking-events et messaging-events
Fournisseur d'email	smtp (Mailpit) — bascule de recette, interface sur http://localhost:8026 (le port 8025 est pris par un autre projet, cf. PR #230)
Jeu d'essai rejoué à	8 septembre 2026, seed-deals.ts — 22 bookings couvrant les 9 statuts
smoke-services.sh	6/6  (build --skip-nx-cache préalable)
Services servis	les six depuis leurs bundles (node --env-file), pas nx serve

Chapitre 4 — Scénarios transverses

22 fiches jouées, 0 ❌. 17 OK, 2 PARTIEL, 3 KO — dont une KO au critère bloquant du cahier.

Fiche	Intitulé	Gravité	Verdict	Pourquoi	Recommandation
API-GW-01	Sonde publique ok	majeure	OK	200, status:"ok", cinq services reachable:true, ms numérique. Cache-Control: no-store posé, cache de 10 s vérifié (deux appels rapprochés → même at). Aucune URL interne ni trace dans le corps.	—
API-GW-02	Sonde down	majeure	OK	message-service coupé → 503, status:"down", l'entrée muette en reachable:false / status:null, les quatre autres intactes. Retour à ok après redémarrage.	—
API-GW-03	Santé passerelle	mineure	OK	200, service:"api-gateway", uptimeSeconds entier, contrôle maintenance.ok:true — et false avec le motif "maintenance (env)" une fois le mode activé.	—
API-GW-04	/health des cinq	mineure	OK	200 pour les cinq, chaque corps portant mongo et redis avec ok / ms / error.	—
API-GW-05	État public de maintenance	mineure	OK	200, {"enabled":false,"message":{"fr":"","en":"","scheduledAt":null}, no-store, sans session.	—
API-GW-06	Écriture refusée en maintenance	majeure	OK	503, corps MAINTENANCE avec messages.fr / .en et retryAfterSeconds:300, en-tête Retry-After: 300. Effet de bord vérifié : aucun favori créé après la levée.	—
API-GW-07	Lectures et exceptions en maintenance	majeure	OK	recherche 200, POST /auth/login 200, /maintenance 200 avec enabled:true. Aucun 503 sur les quatre , ce qui est le point de la fiche.	Le (d) admin a répondu 401 faute de session admin (la 2FA TOTP n'est pas ouvrable en API pure) — à rejouer au cahier Admin.
API-GW-08	Plafonds 100 / 1 000	majeure	OK	RateLimit-Limit : anonyme 100 , jeton valide 1000 , jeton forgé Bearer aaa.bbb.ccc 100 . Franchissement joué : 93×200 puis 17×429 , corps {"error":"Too many requests, please try again later!"} (forme n° 4).	Compteurs remis à zéro par redémarrage de la passerelle (store en mémoire, pas Redis) avant la suite.
API-GW-09	Identifiant de corrélation	majeure	OK	Sans en-tête fourni : UUID v4 généré et renvoyé. Avec recette-api-0001 : le même est renvoyé.	—

Fiche	Intitulé	Gravité	Verdict	Pourquoi	Recommandation
API-GW-10	x-locale pilote le formatage	mineure	PARTIEL	Dates serveur conformes (18 septembre 2026 / September 18, 2026). Mais /messages/quick-replies renvoie le même texte français quel que soit x-locale : le contrôleur lit preferredLocale ?? x-locale (message.controller.ts:29). Contre-épreuve décisive : preferredLocale passé à en → le texte devient "I'm on my way."	Le code a raison, le cahier a tort : la route est documentée « dans la langue du lecteur » et c'est la doctrine du projet (la langue suit le destinataire, pas l'appelant). Corriger l'attendu de la fiche.
API-GW-11	Langue non supportée	mineure	OK	400, details.type:"locale", details.code:"LOCALE_UNSUPPORTED". Contre-épreuve en :200 puis preferredLocale:"en". Remis à fr.	—
API-GW-12	Locales exotiques	mineure	OK	fr-FR, FR, en-US,en;q=0.9, zz, vide → 200 pour les cinq.	—
API-GW-13	details.code arrive au client	bloquante	OK	(a) 409 type:"booking" / code:"FAMILY_REFUSED" · (b) 403 code:"SUDO_REQUIRED", windowMinutes:15 · (c) 400 code:"DELIVERY_CODE_IN_MESSAGE". details présent dans les trois.	La cible (c) du cahier est fausse : bzv-accepted est ACCEPTED et n'a pas encore de deliveryCodeHash (vérifié en base), donc la garde n'a rien à comparer et le 201 était légitime. Rejoué sur bzv-picked (hash présent) → 400 attendu. Remplacer la cible par un deal post-pickup.
API-GW-14	Erreurs par champ	mineure	KO	400 obtenu mais corps {"status":"error","message":"Invalid email format!"} — pas d'objet errors , une seule erreur, message anglais. validateRegistrationData (auth.helper.ts:163-181) valide en séquence et jette au premier échec.	Brancher le schéma Zod RegisterRequestSchema sur le contrôleur, comme le fait déjà message-service (zodErrors()) et comme le prouve deal-service, qui renvoie bien errors: {family, expectedTotalCents}. Incohérence de contrat entre services , pas seulement un écart au cahier.
API-GW-15	Aucune erreur non gérée	bloquante	KO le 08/09 → corrigé le 08/09	Une occurrence trouvée — voir AN0-API-01. Aucun autre corps collecté ne porte la forme n° 2.	Voir la fiche d'anomalie.
API-GW-16	401 sans session	bloquante	OK	401 pour les quatre routes, corps plat {"message":"Unauthorized! Token missing."} — clé unique message, ni status ni details.	—
API-GW-17	403 non partie prenante	bloquante	OK	403 {"status":"error","message":"You are not a party to this deal."}. Zéro donnée du deal dans le corps (recherche de statut, montants, nom du destinataire : 0 occurrence). Contre-épreuve : expéditrice 200, voyageur 200.	—
API-GW-18	404 sans divulgation	bloquante	KO le 08/09 → corrigé le 08/09	Les corps sont irréprochables — (a) 404, (b) 400, (c) 404 PublicNotFound, (d) 404, (e) 404, et un trajet masqué comme un profil privé rendent un corps identique à celui de l'inexistant. Mais le temps de réponse distingue les deux états — voir AN0-API-02.	Voir la fiche d'anomalie.
API-GW-19	409 conflit typé	majeure	PARTIEL	409, details.type:"booking", details.code:"TRANSITION_NOT_ALLOWED" ✓, et allowedActions: [] sur un deal COMPLETED ✓. Mais details.reason est absent : le motif (Action "accept" is not allowed from status COMPLETED.) n'existe que dans le message anglais.	Le cahier interdit lui-même de juger sur le message anglais : ajouter reason (ou from / action) dans details, sinon un client ne peut pas expliquer le refus. À défaut, corriger l'attendu de la fiche.
API-GW-20	Pagination recherche	majeure	OK	Avec limit=1 : page 1 → 1 trajet + nextCursor, page 2 → l'autre, nextCursor:null en dernière page, totalCount stable à 2, aucun identifiant commun . Enveloppe sans success (documenté).	Le jeu d'essai n'expose que 2 trajets recherchables (les autres sont passés) : la fiche est vraie mais peu démonstrative. Prévoir un trajet futur de plus au seed pour paginer sur trois pages.
API-GW-21	Pagination du fil	mineure	OK	Fil rendu du plus ancien au plus récent, curseur = plus ancien message chargé → page antérieure vide , nextCursor:null.	Le fil contenait 3 messages et non 2 : le message de la fiche API-GW-13c y a été ajouté. Sans effet sur le verdict.
API-GW-22	Listes bornées	mineure	OK	3 notifications, unreadCount:3, et ?limit=1000 ignoré (toujours 3) — c'est le point de la fiche.	La borne de 50 n'est pas <i>démontrée</i> faute de volume : à rejouer sur un compte à plus de 50 notifications, ou à borner explicitement au seed.

Anomalies du chapitre 4

ANO-API-01

Fiche : API-GW-15 (révélée en jouant API-GW-20)

Gravité : bloquante au critère du cahier (§4.6 : toute forme n° 2 est bloquante)
– majeure en impact réel : lecture publique, aucune donnée exposée

Environnement : dev 99bc1fc · Atlas development · paiement FAKE · Redpanda actif · Mailpit

Appel exact : curl -s "http://localhost:8080/api/trips/search?limit=1&sort=earliest&cursor=null"

Corps envoyé : – (GET)

Attendu : 400 « Invalid query parameters: cursor », comme pour limit=51

Obtenu : 500 {"status":"error","error":"Something went wrong, please try again!"}
Journal trip-service : PrismaClientKnownRequestError P2023
« Malformed ObjectID: invalid character 'n' ... "null" » dans searchTrips

Reproductible : oui (3/3) – cursor="null", "abc", "pas-un-objectid" ; 200 avec un ObjectId bien formé mais inexistant, et 200 avec cursor vide

Impact : un client qui renvoie littéralement son nextCursor (null sérialisé en "null" – le bug d'intégration le plus courant sur une pagination par curseur) reçoit un 500 au lieu d'une page vide ou d'un 400. Une entrée non validée atteint Prisma ; chaque appel produit une alerte Sentry et pollue la surveillance.

Piste : apps/trip-service/src/dto/trip-search.dto.ts:110 – `cursor: z.string().optional()` sans contrainte de format, alors que `limit` juste en dessous est borné (ligne 111).
Correction d'une ligne : `.regex(/^[\a-f0-9]{24}\$/i)` ou l'ObjectIdSchema partagé.

CORRECTION : 08/09/2026 – apps/trip-service/src/dto/trip-search.dto.ts
`cursor` validé au format ObjectId, la chaîne VIDE valant « pas de curseur » (un client qui envoie toujours `?cursor=` demande la page 1 : la première rédaction du correctif la refusait, régression rattrapée avant la PR et couverte par un test).
Tests : apps/trip-service/src/dto/trip-search.dto.spec.ts (9 cas).

CONTRE-ÉPREUVE : cursor="null" → 400 « Identifiant MongoDB invalide (24 hex attendus) »
cursor="abc" → 400 · cursor="" → 200 (page 1, 1 trajet)
cursor=ObjectId bien formé → 200 · aucune trace P2023 au journal

ÉTAT : CLOSE

ANO-API-02

Fiche	: API-GW-18
Gravité	: bloquante au critère du cahier (« temps de réponse notablement différent ») – à arbitrer : majeure en pratique, le canal exige des mesures répétées
Environnement	: idem
Appel exact	: # trajet masqué par Yamba (hiddenByAdminAt posé en base) contre inexistant curl -s -o /dev/null -w '%{time_total}\n' "\$BASE/trips/<id-masqué>/public" curl -s -o /dev/null -w '%{time_total}\n' "\$BASE/trips/000000000000000000000000/public"
Attendu	: même statut, même corps ET temps indiscernables
Obtenu	: statut et corps IDENTIQUES  – mais, sur 25 mesures chacun : · trajet masqué médiane 32,8 ms (min 30,2 / max 64,7) · trajet inexistant médiane 12,2 ms (min 11,2 / max 20,4) · profil privé médiane 26,5 ms (min 22,8 / max 30,7) · profil inexistant médiane 11,3 ms (min 10,2 / max 15,2) Distributions DISJOINTES dans les deux cas (min du masqué > max de l'inexistant).
Reproductible	: oui (25/25 sur deux routes de deux services différents)
Impact	: un tiers peut énumérer les ressources qui existent mais lui sont cachées – trajets masqués par la modération, profils privés – par la seule mesure du temps, sans jamais lire une donnée. C'est exactement ce que la règle 403/404 cherche à empêcher.
Piste	: trip.controller.ts:1098-1135 – le findUnique charge le trip ET trois jointures (user, avatar, carrierPage) AVANT de tester isDeleted / status / hiddenByAdminAt ligne 1133. L'inexistant ne paie aucune jointure, le masqué les paie toutes. Correctif : porter la visibilité dans le `where` de la requête, en respectant le piège maison Prisma+Mongo sur les champs ABSENTS (`OR: [{hiddenByAdminAt: null}, {hiddenByAdminAt: {isset: false}}]`) Même schéma à corriger sur le profil public (auth-service).
CORRECTION	: 08/09/2026 – la visibilité descend dans la REQUÊTE, des deux côtés : · apps/trip-service/src/lib/public-visibility.rules.ts → publicTripWhere() (publié + non supprimé + non masqué, avec le OR isset:false du piège Mongo) · apps/auth-service/src/utls/public-visibility.ts → publicProfileWhere() (non supprimé + public OU consulté par son propriétaire – D67 1A préservée) Les deux contrôleurs passent de `findUnique` à `findFirst` (un critère non unique est interdit sur findUnique). Le test applicatif est conservé en ceinture et bretelles. Tests : les deux .spec.ts voisins (7 cas).
CONTRE-ÉPREUVE	: mêmes 25 mesures, après correction – · trajet masqué médiane 13,6 ms (min 12,3 / max 29,6) · trajet inexistant médiane 13,5 ms (min 11,8 / max 18,1) · profil privé médiane 12,5 ms (min 10,4 / max 17,6) · profil inexistant médiane 12,3 ms (min 10,6 / max 21,6) Distributions désormais SUPERPOSÉES, corps toujours identiques. Non-régressions vérifiées : trajet publié → 200 · profil masqué vu par un tiers → 404 · profil masqué vu par SON propriétaire → 200 (D67 1A).
ÉTAT	: CLOSE

ANO-API-03

Fiche	: API-GW-14
Gravité	: mineure
Appel exact	: curl -s -X POST "\$BASE/auth/register" -H 'Content-Type: application/json' \ -H 'x-locale: fr' -d '{"firstName":"A","lastName":"B","email":"pas-un-email", "password":"123","termsAccepted":true,"termsVersion":"2026-06-01", "privacyVersion":"2026-06-01"}'
Attendu	: 400 + objet `errors` au premier niveau ({ "email": "...", "password": "..." })
Obtenu	: 400 {"status":"error","message":"Invalid email format!"} – aucun `errors`, seule la PREMIÈRE erreur rencontrée est signalée
Reproductible	: oui
Impact	: le front ne peut pas afficher les erreurs sous chaque champ à l'inscription ; l'utilisateur corrige une erreur à la fois. Incohérence avec deal-service et message-service, qui renvoient bien un `errors` champ par champ.
Piste	: apps/auth-service/src/utls/auth.helper.ts:163-181 (validation séquentielle legacy). Le contrat Zod existe déjà mais ne garde pas encore ces contrôleurs (dette connue, member-auth.schema.ts).
CORRECTION	: 08/09/2026 – la règle sort du contrôleur : apps/auth-service/src/utls/ registration-rules.ts (`collectRegistrationErrors`), pure, sans Redis ni Prisma, testable telle quelle comme password-rules et otp-policy à côté. TOUS les champs sont examinés ; l'erreur porte `details.errors` { champ → code } (REQUIRED, INVALID_FORMAT, TERMS_NOT_ACCEPTED). Le mot de passe garde ses règles propres : quand il est le SEUL fautif, son erreur typée (`details.type: "password" + code) est relayée telle quelle.
CONTRE-ÉPREUVE	: email invalide + mot de passe court → 400 { "errors": { "email": "INVALID_FORMAT", "password": "PASSWORD_TOO_SHORT" } } mot de passe seul fautif → 400 { "type": "password", "code": "PASSWORD_TOO_SHORT" }
ÉTAT	: CLOSE

ANO-API-04	
Fiche	: API-GW-19
Gravité	: mineure
Appel exact	: curl -s -X POST "\$BASE/deals/<deal COMPLETED>/accept" -b carrier.txt \
	-H 'Content-Type: application/json' -d '{"charterAccepted":true}'
Attendu	: 409 + details.type + details.code + details.reason (motif de la machine à états)
Obtenu	: 409 {"status":"error","message":"Action \"accept\" is not allowed from status COMPLETED.", "details":{"type":"booking", "code":"TRANSITION_NOT_ALLOWED"}} – le motif n'existe que dans le `message` anglais, `details.reason` absent
Reproductible	: oui
Impact	: le cahier interdit de juger un refus sur le message anglais ; un client ne peut donc pas expliquer POURQUOI la transition est refusée sans parser de l'anglais.
Piste	: la machine à états connaît `from`, `to` et `action` – les exposer dans `details`.

Écarts du cahier (à corriger dans le cahier, pas dans le code)

1. **API-GW-13 (c)** vise `bzv-accepted`, un deal **ACCEPTED sans code de livraison** : la garde ne peut pas se déclencher et le 201 obtenu est correct. Cible à remplacer par un deal post-pickup (`bzv-picked`), où le refus 400 `DELIVERY_CODE_IN_MESSAGE` est bien au rendez-vous.
2. **API-GW-10** attend des réponses rapides traduites par `x-locale` alors que le service suit — délibérément et conformément à la doctrine — la langue **du lecteur** (`preferredLocale`).
3. **API-GW-20** suppose plus de deux trajets recherchables ; le jeu d'essai n'en publie que deux dans le futur.

Commentaire de chapitre

Les gardes qui protègent l'argent et les données tiennent : `details.code` arrive au client dans les trois familles de refus (API-GW-13, bloquante), le 401 est plat et muet, le 403 ne laisse **rien** filtrer du deal, la maintenance bloque les écritures sans jamais bloquer la connexion, et le limiteur distingue un jeton **vérifié** d'un simple cookie présent — la correction A147 tient.

Les deux vrais défauts sont d'une autre nature : ce sont des **fuites par le canal secondaire** et des **entrées non validées**. ANO-API-02 est le plus intéressant du chapitre : le corps est parfait, le statut est parfait, et pourtant la ressource cachée se trahit par 20 ms d'écart, sur deux services indépendants — signe que la règle « vérifier la visibilité dans la requête, pas après » n'est écrite nulle part. ANO-API-01 rappelle qu'un paramètre de pagination est une entrée utilisateur comme une autre.

Aucune de ces deux anomalies n'est un défaut de conception : les deux se corrigent dans une PR courte, respectivement une ligne de schéma et deux clauses `where`.

Suivi des anomalies

L'historique est conservé : un KO reste écrit KO, sa correction s'ajoute en dessous.

Anomalie	Fiche	Gravité	Constat	État	Correction
ANO-API-01	API-GW-15 / 20	bloquante (critère cahier)	08/09/2026	close	validation du curseur + test — même campagne
ANO-API-02	API-GW-18	bloquante (critère cahier)	08/09/2026	close	visibilité dans le <code>where</code> , deux services + tests
ANO-API-03	API-GW-14	mineure	08/09/2026	close	règle pure <code>collectRegistrationErrors</code> + tests
ANO-API-04	API-GW-19	mineure	08/09/2026	close	fermée au chapitre 9 — voir la ligne en fin de tableau
ANO-API-10	API-TRIP-03	mineure	08/09/2026	close	<code>.catch("all")</code> sur le filtre <code>mode</code>
ANO-API-11	API-TRIP-01 / 04	majeure	08/09/2026	close	invariant à la publication + exclusion retirée + seed
ANO-API-12	API-DEAL-02	majeure	08/09/2026	close	valeur déclarée confrontée aux plafonds avant l'autorisation
ANO-API-13	API-DEAL-22	bloquante	08/09/2026	close	champ corrigé + garde-fou <code>select</code> systémique sur deal-service
ANO-API-14	API-DEAL-20	majeure	08/09/2026	close	<code>z.partialRecord</code> — piège d'exhaustivité de Zod 4
ANO-API-15	API-DEAL-06 / 15	majeure	08/09/2026	close	destinataire minimisé ; verrou typé avec son horizon

Anomalie	Fiche	Gravité	Constat	État	Correction
ANO-API-16	API-NOTIF-01 / 04	bloquante	08/09/2026	close	types conversation.* au contrat + lecture robuste
ANO-API-17	API-NOTIF-04	majeure	08/09/2026	close	gabarits d'email embarqués, chemin indépendant du cwd
ANO-API-18	API-SEC-14	bloquante	08/09/2026	close	connexion à temps constant (membre ET admin)
ANO-API-05	API-AUTH-03 / 05	majeure	08/09/2026	close	409 / 401 / 429, classes d'erreur enrichies
ANO-API-06	API-AUTH-09	bloquante	08/09/2026	close	liste blanche + test lisant le schéma Prisma
ANO-API-07	API-AUTH-11 / 12 / 13	majeure	08/09/2026	close	jti dans le jeton d'accès — registre D75 candidate
ANO-API-08	API-AUTH-14	bloquante	08/09/2026	close	envoi détaché de la réponse (forgot ET resend)
ANO-API-09	API-AUTH-22 / 24 / 25	bloquante	08/09/2026	close	champ corrigé + test lisant le schéma ; API-AUTH-24 enfin jouée (OK)
ANO-API-19	API-IDEM-07	bloquante	08/09/2026	close	conflit d'écriture Mongo (P2034) rejoué, jamais rendu en 500
ANO-API-20	API-IDEM-08	majeure	08/09/2026	close	tout refus de message-service porte un details.code + garde-fou lisant les sources
ANO-API-21	API-IDEM-09	majeure	08/09/2026	close	protection P2034 remontée au writer central applyBookingTransition
ANO-API-04	API-GW-19	mineure	08/09/2026	close	le 409 de transition dit enfin depuis quel statut ; tout refus de deal-service porte un code
ANO-API-23	hors cahier (solde D-5)	bloquante	08/09/2026	close	les deux pages publiques répondaient 404 (22 comptes sur 26, 24 trajets sur 37) : champ requis ABSENT, données réparées + filtres remis en égalité

Chapitre 8 — aucune anomalie.

Vingt-trois anomalies, vingt-trois closes — la vingt-troisième trouvée après la campagne, en soldant la dette D-5. Les onze bloquantes sont toutes fermées **et contre-évaluées** : le critère de sortie n° 1 du §9.3 (« zéro anomalie bloquante ouverte ») est tenu. Aucune anomalie n'a été « acceptée avec contournement ».

ANO-API-04, dernière ouverte à la fin du chapitre 7, a été fermée au **chapitre 9** : le 409 `TRANSITION_NOT_ALLOWED` porte désormais le statut d'où l'action a été tentée et ceux d'où elle reste possible, et les quarante et un refus métier de deal-service portent leur code.

Référence de tests après corrections : trip-service 209 → **221**, auth-service 183 → **192** (plateforme 860 → **877**), CLAUDE.md mis à jour.

Chapitre 5.1 — auth-service (terminé)

34 fiches : 32 jouées, 2 ❌. 24 OK, 8 KO — dont **trois bloquantes** (une close, deux ouvertes). Les deux ❌ (API-AUTH-24 et 25) ne sont pas un choix : elles portent sur le contenu de l'export, et l'export lui-même échoue (ANO-API-09).

Fiche	Intitulé	Gravité	Verdict	Pourquoi	Recommandation
API-AUTH-01	Démarrer une inscription	majeure	OK	200, verificationToken de 64 caractères. Rien en base : POST /auth/login sur ce compte répond 401. Le code à six chiffres est arrivé dans Mailpit, en français, expéditeur no-reply@yamba.test, objet « Ton code d'activation Yamba » (tutoiement conforme aux décisions du 03/09).	—
API-AUTH-02	Consentement obligatoire	majeure	OK	termsAccepted:false → 400 ; sans termsVersion → 400 avec un motif distinct.	—
API-AUTH-03	Email déjà pris	majeure	KO	details.code:"EMAIL_ALREADY_USED" et details.type:"register" sont bien là, mais le statut est 400 , pas 409.	Voir ANO-API-05 — l'OpenAPI documente 409.
API-AUTH-04	Mot de passe faible	mineure	OK	400, details.type:"password", details.code:"PASSWORD_TOO_SHORT", details.field:"password".	—

Fiche	Intitulé	Gravité	Verdict	Pourquoi	Recommandation
API-AUTH-05	Code faux, compteur	majeure	KO	La mécanique est exacte : <code>attemptsLeft 4→3→2→1</code> , puis <code>OTP_INVALIDATED</code> avec <code>locked:true</code> et <code>lockUntilSeconds:60</code> (premier palier), puis <code>OTP_LOCKED</code> — et le bon code est refusé pendant le verrou, ce qui est le comportement voulu. Mais les statuts sont 400 partout, au lieu de 401 (code faux) et 429 (verrou).	Voir ANO-API-05 .
API-AUTH-06	Bon code, sans session	majeure	OK	201, <code>{"success":true}</code> , et grep -c access_token = 0 : aucune session n'est ouverte à la vérification.	—
API-AUTH-07	Jetons en cookies seulement	bloquante	OK	200. Le corps ne porte que <code>message</code> et <code>user</code> (id, email, firstName, lastName, roles) — aucun <code>accessToken</code> , <code>refreshToken</code> ni <code>passwordHash</code> . Deux <code>Set-Cookie HttpOnly + SameSite=Lax : access_token Max-Age 900 s, refresh_token Max-Age 2 592 000 s</code> (30 j avec <code>rememberMe</code>). Les anciens cookies sont expirés avant d'être réécrits.	—
API-AUTH-08	401 indistinguable	bloquante	OK	Mot de passe faux et compte inconnu : 401 tous les deux , corps rigoureusement identiques (<code>Invalid email or password</code>).	—
API-AUTH-09	<code>/auth/me</code> sans secret	bloquante	KO le 08/09 → corrigé le 08/09	<code>passwordHash</code> est bien retiré, mais quatre champs TOTP sortent : <code>totpSecretEncrypted</code> , <code>totpBackupCodeHashes</code> , <code>totpEnabledAt</code> , <code>totpLastUsedStep</code> — plus les champs de modération <code>suspensionProposed*</code> .	Voir ANO-API-06 .
API-AUTH-10	Rotation de session	majeure	OK	<code>POST /auth/refresh</code> → 200 <code>{success:true}</code> sans jeton dans le corps ; le pot mis à jour ouvre <code>/auth/me</code> (200) ; l' ancien pot répond 401 (« Session expired or invalid »). Un jeton de rafraîchissement ne sert bien qu'une fois.	—
API-AUTH-11	Mes appareils	majeure	KO	La liste est juste (deux sessions, une seule <code>current:true</code> , <code>device / ip / dates</code>) et la révocation répond 200 <code>{ok:true}</code> — mais la session révoquée continue d'ouvrir /auth/me (200) . Seul <code>/auth/refresh</code> est coupé (401).	Voir ANO-API-07 . Écart de cahier au passage : la réponse s'appelle <code>items[]</code> , pas <code>sessions[]</code> .
API-AUTH-12	Mot de passe révoque les sessions	majeure	KO	La chaîne complète marche : <code>sudo/request</code> → code lu dans Mailpit → <code>sudo/verify</code> (200, <code>active:true</code>) → changement accepté ; l'ancien mot de passe répond 401, le nouveau 200. Mais les autres sessions gardent l'accès : <code>/auth/me</code> répond encore 200 sur l'autre appareil, seul <code>/auth/refresh</code> est coupé.	Voir ANO-API-07 — c'est la troisième surface touchée. Écart de cahier : <code>sudo/verify</code> attend le champ <code>code</code> , pas <code>otp</code> .
API-AUTH-13	Déconnexion	majeure	KO	200 puis 401, et le pot ne porte plus de cookie — mais avec une copie du pot prise avant le logout, <code>/auth/me</code> répond toujours 200. Seul le rafraîchissement est révoqué : la déconnexion est côté client, pas côté serveur.	Voir ANO-API-07 .
API-AUTH-14	Mot de passe oublié muet	bloquante	KO	Corps et statut rigoureusement identiques pour un compte existant et un compte inconnu ✅ — mais le temps trahit : 95,7 ms contre 18,8 ms (15 mesures, min 87,0 > max 23,6).	Voir ANO-API-08 .
API-AUTH-15	Profil éditable, slug stable	mineure	OK	200 sur les champs du membre, <code>publicSlug</code> identique avant et après .	La fiche envoie <code>displayName</code> et <code>bio</code> sur une Expéditrice : ces champs appartiennent à la page Voyageur, d'où un 400 <code>NO_CARRIER_PAGE</code> champ par champ — refus correct. Rejoué sur un Voyageur : 200. Cible à corriger dans le cahier.
API-AUTH-16	Date de naissance invalide	mineure	OK	400 avec <code>errors: {"birthDate":"IN_THE_FUTURE"}</code> puis <code>"T00_YOUNG"</code> . Les erreurs sont ici champ par champ — ce qui confirme que ANO-API-03 est propre à <code>/auth/register</code> et à sa validation historique, pas au service entier.	—
API-AUTH-17	Avatar signé, URL gardée	majeure	OK	<code>/uploads/imagekit-auth</code> : 200 avec <code>token</code> (36), <code>signature</code> (40), <code>expire</code> , plus <code>publicKey</code> et <code>urlEndpoint</code> — l'environnement ImageKit est bien configuré. La garde tient : une URL étrangère est refusée en 400 (« must belong to Yamba's media endpoint »).	Étapes 2 et 3 (téléversement réel) ➡ : elles écrivent un fichier chez un prestataire tiers, ce qui n'a pas sa place dans une campagne API. À couvrir au cahier Web.

Fiche	Intitulé	Gravité	Verdict	Pourquoi	Recommandation
API-AUTH-18	Profil public / masqué	bloquante	OK	Jamais 401 : sans session, avec un tiers, avec un jeton forgé, sur /reviews et /trips → 200 partout quand le profil est public. Masqué : un tiers reçoit 404 , le propriétaire 200 avec hidden:true . La seule asymétrie autorisée est bien la seule présente — et elle valide au passage le correctif ANO-API-02 en conditions réelles.	—
API-AUTH-19	Abonnements	mineure	OK	Abonnement 200, rejeu sans doublon (la liste ne porte qu'une entrée), désabonnement 200 et liste vide ensuite, refus 400 sur soi-même.	—
API-AUTH-20	Préférences et consentement	mineure	OK	200 sur analyticsOptIn et messagingReminderEmails . Effet de bord vérifié en base : une ligne ConsentLog COOKIES version cookies-2026-09 est écrite — le consentement est tracé, pas seulement stocké.	—
API-AUTH-21	Geste sensible sans fenêtre	bloquante	OK	/auth/me/sudo → {active:false, expiresAt:null} ; export, effacement et lien Stripe → 403 SUDO_REQUIRED avec windowMinutes:15 .	—
API-AUTH-22	Ouvrir la fenêtre puis exporter	bloquante	KO	La fenêtre s'ouvre parfaitement (code lu dans Mailpit, verify 200, active:true avec expiresAt à +15 min) — puis l'export répond 500 .	Voir ANO-API-09 .
API-AUTH-23	La fenêtre est liée à UNE session	bloquante	OK	La session d'origine a active:true ; l'autre appareil du même membre voit active:false et reçoit 403 SUDO_REQUIRED sur l'export. Un appareil compromis n'hérite de rien.	—
API-AUTH-24	Export sans données d'autrui	bloquante	⚡	Non jouable : l'export échoue (ANO-API-09). À rejouer en priorité après correction — c'est une bloquante qui n'a pas encore de verdict.	—
API-AUTH-25	Export trop fréquent	mineure	⚡	Idem.	—
API-AUTH-26	Bloqueurs d'effacement	majeure	OK	200, blockers: [ACTIVE_DEAL, PENDING_REQUEST, RETENTION_HELD, ADMIN_ACCOUNT] et counts détaillés ; la liste fermée est respectée.	—
API-AUTH-27	Effacement bloqué	majeure	OK	409, code:"ERASURE_BLOCKED" à la racine comme documenté, avec blockers et counts .	—
API-AUTH-28	Effacement effectif	bloquante	OK	200 {success:true, erased:true} puis 401 ACCOUNT_DELETED : la session tombe immédiatement. En base: email et emailNormalized → erased+<id>@anonymised.invalid, publicSlug → deleted-<id>, firstName/lastName → « Membre »/« supprimé », isDeleted:true, deletedAt posé, ligne DataRequest ERASURE / DONE . Aucun champ unique à null (phoneE164 l'est, mais il n'est qu'indexé — vérifié au schéma).	—
API-AUTH-29	Alertes de route	mineure	OK	201 avec isActive:true et expiresAt à six mois jour pour jour ; lecture 200 ; suppression 200.	—
API-AUTH-30	Gardes des alertes	majeure	OK	Origine = destination → 400. Plafond : 20 créations acceptées, la 21e refusée en 400 avec un message explicite. Nettoyage vérifié (0 restante).	—
API-AUTH-31	Signaler un trajet	majeure	OK	201 avec reportId et createdAt .	—
API-AUTH-32	Gardes du signalement	majeure	OK	Les quatre gardes tiennent : 400 OWN_TARGET , 400 REASON_NOT_ALLOWED (motif d'une autre cible), 404 pour un slug inexistant, 409 sur le doublon du même auteur.	—

Fiche	Intitulé	Gravité	Verdict	Pourquoi	Recommandation
API-AUTH-33	Onboarding Voyageur	majeure	OK	(1) 200, onboardingStep passe à STRIPE ; (2) 200 avec une url Stripe ; (3) 200 status:"pending" et les trois drapeaux à false ; (4) 200.	Deux précisions pour le cahier : le rôle CARRIER n'apparaît dans rôles qu'après renouvellement de la session (les rôles vivent dans le jeton) ; et l'étape (4) réussit malgré un Stripe incomplet — ce n'est pas un défaut, la garde D31 est en aval (deal-lifecycle.service.ts:151 refuse l'acceptation d'un deal sans stripeOnboardingComplete ni stripeChargesEnabled).
API-AUTH-34	Tableau de bord sans compte Connect	mineure	OK	403 SUDO_REQUIRED sans fenêtre, puis 409 STRIPE_ACCOUNT_MISSING une fois la fenêtre ouverte : l'ordre des gardes est le bon.	—

Anomalies du chapitre 5.1

ANO-API-05

Fiches : API-AUTH-03, API-AUTH-05

Gravité : majeure

Constat : auth-service répond 400 pour des refus que son PROPRE contrat OpenAPI documente autrement.

- POST /auth/register, email déjà pris → 400 obtenu, 409 documenté
- POST /auth/register/verify, code faux → 400 obtenu, 401 documenté
- POST /auth/register/verify, verrouillé → 400 obtenu, 429 documenté (statuts lus dans apps/auth-service/openapi.json, généré depuis les contrats Zod – A145)

Reproductible : oui, systématique

Impact : le front web actuel décide sur `details.code` et n'est pas gêné. Un client GÉNÉRÉ depuis l'OpenAPI – le client mobile de D36 est le cas prévu – code des branches 409 et 429 qui ne seront jamais atteintes, et traite ces refus comme de simples erreurs de saisie. Un 429 manquant prive aussi les clients et les proxys de la sémantique « ralentis », la seule qu'un intermédiaire comprend sans lire le corps.

Piste : les contrôleurs d'inscription lèvent tous une ValidationError (400). Il existe déjà des erreurs typées côté @packages/error-handler ; le code de refus (`EMAIL\_ALREADY\_USED`, `OTP\_INCORRECT`, `OTP\_LOCKED`) est présent et juste – seul le statut porté par l'erreur est à corriger.

CORRECTION : 08/09/2026 – les trois classes de @packages/error-handler acceptent désormais un `details` (AuthError, RateLimitError, ConflictError) : sans lui, il fallait passer par ValidationError, donc répondre 400, pour ne pas perdre le code métier. Puis EMAIL_ALREADY_USED → ConflictError (409, trois sites) ; OTP_INCORRECT → AuthError (401) ; OTP_INVALIDATED et OTP_LOCKED → RateLimitError (429), les deux portant un verrou.

CONTRE-ÉPREUVE : email déjà pris → 409 `EMAIL\_ALREADY\_USED` ; essais OTP → 401, 401, 401 (compteur 4→1), puis 429 OTP_INVALIDATED, puis 429 OTP_LOCKED. Codes, compteurs et verrous inchangés.

ÉTAT : CLOSE

ANO-API-06

Fiche	: API-AUTH-09
Gravité	: BLOQUANTE
Appel exact	: curl -s -b shipper.txt "\$BASE/auth/me" jq '.user keys'
Attendu	: le document User SANS passwordHash ni aucun secret (fuites: [])
Obtenu	: 200, `passwordHash` bien retiré – mais le corps porte `totpSecretEncrypted`, `totpBackupCodeHashes`, `totpEnabledAt`, `totpLastUsedStep`, ainsi que `suspensionProposedReason`, `suspensionProposedLevel`, `suspensionProposedByAdminId`, `invitedByAdminId`, `emailSuppressedReason`, `emailNormalized`.
PREUVE	: sur un compte du jeu d'essai sans 2FA ces champs valent null / [] – ce qui pouvait faire croire à un faux positif. Des valeurs factices ont donc été posées en base, puis retirées : /auth/me a bien renvoyé au membre `"totpSecretEncrypted":"FAKE-AES-GCM:..."`, les deux hachages de codes de secours, et `"suspensionProposedReason":"Motif interne rédigé par un administrateur..."`. Base restaurée après la mesure.
Reproductible	: oui
Impact	: 1. Le secret du second facteur (chiffré) et les hachages des codes de secours d'un compte ADMINISTRATEUR sont lisibles par toute session de ce compte : un vol de cookie ou un XSS donne de quoi attaquer le 2FA hors ligne, alors que D54 en fait la garde des accès admin. La clé de déchiffrement a un repli de développement hors production – deux moitiés qu'il vaut mieux ne pas laisser voyager ensemble. 2. Un membre lit le MOTIF de modération rédigé contre lui et l'identité de l'administrateur, avant même toute décision de suspension.
Piste	: apps/auth-service/src/controller/auth.controller.ts:672 `const { passwordHash, ...safeUser } = fullUser;` C'est le `spread + delete` que les règles non négociables du projet interdisent explicitement : tout champ ajouté au modèle User part vers le client sans que personne ait à y penser – c'est ainsi que les champs TOTP de D54, écrits bien après, sont sortis. Correctif : une liste blanche explicite (`select` Prisma ou DTO), et un test qui échoue si un champ inconnu apparaît dans la réponse.
CORRECTION	: 08/09/2026 – apps/auth-service/src/utils/me-projection.ts `ME_USER_SELECT` (liste blanche passée en `select` Prisma) et `ME_EXCLUDED_FIELDS`, où chaque exclusion porte SA RAISON en clair. Le `spread + delete` disparaît du contrôleur. Sont conservés à dessein : `totpEnabledAt` (dire « la 2FA est active » ne livre rien), `suspendedAt` / `suspensionReason` / `suspensionUntil` (une sanction PRONONCÉE est notifiée au membre, contrairement à une proposition). Aucun des champs retirés n'était utilisé par user-ui ni admin-ui (vérifié par recherche avant correction).
TEST	: apps/auth-service/src/utils/me-projection.spec.ts (5 cas). Le premier LIT prisma/schema.prisma et échoue tant qu'un champ du modèle `User` n'est pas classé – renvoyé ou explicitement exclu. Un test qui se contenterait de vérifier l'absence de `totpSecretEncrypted` raterait le prochain champ sensible ; celui-ci ne peut pas le rater.
CONTRE-ÉPREUVE	: mêmes valeurs factices reposées en base, /auth/me renvoie désormais `fuites: []` et aucun des champs sensibles n'apparaît, tandis que `totpEnabledAt`, `avatar`, `carrierPage`, `roles`, `adminRoles`, `accountStatus`, `preferredLocale` restent servis. Base restaurée.
ÉTAT	: CLOSE

ANO-API-07

Fiche : API-AUTH-11

Gravité : majeure

Appel exact : DELETE "\$BASE/auth/me/sessions/<jti>" -b shipper.txt → 200 {ok:true}
 GET "\$BASE/auth/me" -b autre-appareil.txt → 200 (attendu 401)
 POST "\$BASE/auth/refresh" -b autre-appareil.txt → 401 (correct)

Attendu : « la révocation est immédiate et côté serveur »

Obtenu : la révocation coupe le RAFFRAÎCHISSEMENT mais pas l'accès en cours : le jeton d'accès reste accepté jusqu'à son expiration naturelle (15 min).

Reproductible : oui

Impact : « Couper cet appareil » ne coupe rien pendant un quart d'heure. C'est exactement la fonction qu'un membre utilise quand il PENSE avoir été compromis : pendant 15 minutes, l'attaquant garde la lecture de son profil, de ses deals et de ses messages. Même limite pour le changement de mot de passe (API-AUTH-12, à jouer).

Piste : packages/middleware/isAuthenticated.ts vérifie la signature du jeton puis lit l'utilisateur en base – ce qui rend d'ailleurs la SUSPENSION de compte immédiate, elle – mais ne consulte aucune liste de révocation. Le `jti` est déjà dans le jeton (la liste des sessions l'affiche) et Redis est déjà là : un test d'existence par requête authentifiée, avec un TTL calé sur la durée de vie du jeton d'accès, suffit. À défaut, raccourcir le jeton d'accès.

ÉTENDUE : trois surfaces mesurées, toutes avec le même symptôme (accès conservé, rafraîchissement coupé) –
 · API-AUTH-11 « couper cet appareil »
 · API-AUTH-13 « se déconnecter » (avec une copie du pot prise avant)
 · API-AUTH-12 « changer de mot de passe », qui doit couper les autres sessions

La gravité tient à ce troisième cas : on change son mot de passe précisément quand on se croit compromis, et l'intrus garde la lecture un quart d'heure.

CORRECTION : 08/09/2026 – décision d'architecture proposée au registre en **D75 (candidate)**, comme l'exige la règle du projet. Le jeton d'accès porte désormais le `jti` de sa session (émission ET rotation), et `isAuthenticated` vérifie que `refresh\_jti:<userId>:<jti>` existe encore – la clé que auth-service posait déjà. La décision est isolée dans packages/middleware/session-revocation.ts : pas de `jti` (jeton émis avant le déploiement) → accepté jusqu'à expiration ; Redis muet → laissé passer (une panne de cache ne déconnecte pas la plateforme, et le compte suspendu ou effacé reste refusé par la lecture Mongo).

CONTRE-ÉPREUVE : les trois surfaces, la session courante restant vivante à chaque fois –
 · couper un appareil → l'autre session : 401 SESSION_REVOKED
 · se déconnecter → le pot d'AVANT : 401 SESSION_REVOKED
 · changer de mot de passe → l'autre appareil : 401 SESSION_REVOKED

ÉTAT : CLOSE

ANO-API-08

Fiche : API-AUTH-14

Gravité : BLOQUANTE (« toute différence est bloquante », §5.1.3)

Appel exact :

```
for i in $(seq 1 15); do curl -s -o /dev/null -w '%{time_total}\n' \
-X POST "$BASE/auth/password/forgot" -H 'Content-Type: application/json' \
-d '{"email":"<adresse>}'; done | sort -n
```

Attendu : aucune différence observable entre un compte existant et un compte inconnu

Obtenu : statut 200 et corps identiques ✅ (« If an account exists, an OTP has been sent to the email. ») – mais, sur 15 mesures :

- compte existant médiane 95,7 ms (min 87,0 / max 113,6)
- compte inexistant médiane 18,8 ms (min 12,1 / max 23,6)

Distributions DISJOINTES : un seul appel suffit à trancher.

Reproductible : oui (15/15)

Impact : énumération de comptes. Savoir qu'une adresse a un compte Yamba est une donnée personnelle en soi, et c'est le point de départ classique du bourrage d'identifiants. Le soin pris à rendre le corps identique est annulé par la mesure du temps – c'est le même motif qu'ANO-API-02, sur une autre surface.

Piste : l'envoi de l'email est fait DANS la requête (SMTP synchrone) : le compte existant paie l'aller-retour au serveur d'envoi, l'inconnu ne paie rien. Correctif : répondre d'abord, envoyer ensuite (l'envoi ne conditionne pas la réponse, qui est volontairement muette) – ou aligner les deux chemins. Le même raisonnement vaut pour `/auth/register/resend` et pour tout point d'entrée public qui envoie un email selon l'existence d'un compte.

CORRECTION : 08/09/2026 – compteurs anti-abus ET envoi partent en arrière-plan dans `envoyerSansRienReveler` : la réponse ne dépend plus de rien de ce qui suit, ni par son corps, ni par son statut, ni par son temps. Appliqué à /auth/password/forgot ET /auth/password/resend, qui portait la même fuite. Effet voulu : un compte existant en cooldown recevait une erreur là où un compte inconnu recevait 200 – cette seconde fuite, non temporelle, se referme aussi (le cooldown est désormais journalisé).

CONTRE-ÉPREUVE : 15 mesures – existant 21,6 ms (13,9 / 41,3), inexistant 19,3 ms (12,9 / 24,8) : plages largement superposées. Corps identiques, et les emails arrivent toujours dans Mailpit.

ÉTAT : CLOSE

ANO-API-09

Fiche : API-AUTH-22 (bloque aussi API-AUTH-24 et 25)

Gravité : BLOQUANTE

Appel exact : `curl -s -X POST "$BASE/auth/me/data-export" -b shipper.txt` (fenêtre sudo ouverte)

Attendu : 200, l'export des données du membre, en-tête Content-Disposition, et une ligne DataRequest de type EXPORT

Obtenu : `**500** {"status":"error","error":"Something went wrong, please try again!"}`
 Journal auth-service : PrismaClientValidationError sur db.booking.findMany() dans buildDataExport – « Unknown field `ticketNumber` for select statement on model `Booking` ».

Reproductible : oui, systématique – la fonction ne peut PAS marcher

Cause : apps/auth-service/src/services/privacy.service.ts:155 sélectionne `ticketNumber` sur `Booking`. Vérifié au schéma : `**`Booking` n'a pas ce champ**` (il porte `disputeTicket`, « copie du `Dispute.ticketNumber` ») ; `ticketNumber` appartient au modèle `Dispute`. La ligne 180 lit ensuite ``b.ticketNumber``.

Impact : le droit d'accès du RGPD (D63) est **entièrement inopérant** : aucun membre ne peut obtenir ses données. C'est une obligation légale, et le geste est mis en avant dans l'interface. S'y ajoute un 500 de forme n° 2, que le \$4.6 qualifie de bloquant à lui seul.

Pourquoi les tests ne l'ont pas vu : privacy.service.spec.ts existe et passe – il injecte un faux Prisma, qui ne valide aucun nom de champ. C'est le piège déjà consigné au chapitre 100 de l'apprentissage (« ces objets ne signalent jamais ce qui leur manque »), rencontré ici dans l'autre sens : le mock accepte un champ qui n'existe pas.

Piste : corriger le select (`disputeTicket`) et la projection, puis se donner un garde-fou qui ne dépende pas du mock – par exemple un test qui confronte les champs demandés au modèle de prisma/schema.prisma, comme le fait déjà me-projection.spec.ts depuis ANO-API-06.

CORRECTION : 08/09/2026 – `ticketNumber` → `disputeTicket` dans le select et dans la projection (privacy.service.ts:155 et 180).

TEST : apps/auth-service/src/services/privacy-export-fields.spec.ts – il ne mocke rien : il LIT le source de l'export, en extrait chaque ``db.<modèle>... select`` et confronte les champs à prisma/schema.prisma. Vérifié qu'il échoue bien quand on réintroduit le bug (« `Booking.ticketNumber` »).

CONTRE-ÉPREUVE : export → 200, 14 269 octets, en-tête `content-disposition: attachment; filename="yamba-mes-donnees-2026-09-08.json"`, 20 rubriques.
****API-AUTH-24**, la fiche bloquante restée sans verdict, est enfin jouable et **PASSE** : code de livraison absent, VOYAGEUR (l'autre partie) totalement absent – ni nom, ni email, ni téléphone –, signalements visant le membre et dossiers de médiation absents. Les destinataires présents sont ceux que le membre a saisis lui-même.

ÉTAT : CLOSE

Écarts du cahier (chapitre 5.1)

4. **API-AUTH-11** annonce une réponse `{ "sessions": [...] } ; l'API renvoie { "items": [...] }.`
5. **API-AUTH-12** : `sudo/verify` attend le champ `code`, le cahier écrit `otp`.
6. **API-AUTH-15** applique `displayName` et `bio` à une **Expéditrice** : ces champs sont ceux de la page Voyageur (refus correct `NO_CARRIER_PAGE`).
7. **API-AUTH-33** : le rôle `CARRIER` n'apparaît qu'après **renouvellement de la session**.
8. **API-AUTH-11** annonce `sessions[]`, l'API renvoie `items[]`.
9. **API-AUTH-01** cite un corps `"OTP sent to your email..."` ; le service dit « OTP sent to email. Please verify your account. » (sans portée : on ne juge jamais sur le message).

Challenge expert — auth-service (au-delà des anomalies de recette)

La recette juge des fiches ; ce qui suit est un regard d'ingénierie sur le service tel qu'il est apparu pendant la campagne. **Rien n'est corrigé ici** : ce sont des propositions, classées par rapport valeur / risque, à arbitrer.

1. `isAuthenticated` charge tout le document membre, à chaque requête, sur les six services

```
const user = await prisma.user.findUnique({ where: { id: decoded.id } }); // 60 champs
```

Le middleware n'utilise que `isDeleted`, `accountStatus` et `roles`. Il rapatrie pourtant les **soixante** champs du modèle depuis Atlas (distant), à **chaque requête authentifiée de chaque service** — `passwordHash` et `totpSecretEncrypted` compris, qui transitent donc en mémoire dans trip-service, deal-service, message-service et notification-service sans qu'aucun n'en ait l'usage. C'est la même racine que ANO-API-06 : charger tout par défaut.

Proposition — un `select` explicite. Attention, ce n'est pas une correction de trois lignes : `req.user` est exposé aux contrôleurs, qui lisent aussi `email` (11 usages), `preferredLocale` (7), `firstName` (6), `emailNormalized` (4), `lastName` (2), `analyticsOptIn` (1) et — le point dur — `passwordHash` (3 usages). La bonne cible est un `select` d'une dizaine de champs, les trois appelants de `passwordHash` relisant l'utilisateur eux-mêmes. Gain : moins de trafic sur le chemin le plus chaud de la plateforme, et plus aucun secret hors d'auth-service. **PR dédiée**, avec la liste des usages en preuve.

2. Aucune protection anti-force brute par compte sur `/auth/login`

L'OTP a des paliers de verrou soignés (1 min → 30 min → 24 h) et une alerte de sécurité par email. La **connexion par mot de passe**, elle, n'a rien : seul le limiteur de la passerelle protège, à 100 requêtes / 15 min **par IP**. Un attaquant qui vise un compte précis depuis quelques adresses dispose donc de plusieurs centaines d'essais par heure, et le titulaire n'est jamais prévenu.

Proposition — réutiliser la mécanique OTP, qui existe déjà : compteur par `emailNormalized`, paliers, et email d'alerte au titulaire au deuxième palier. Le refus doit rester **indistinguable** (`ANO-API-08` vient de rappeler que la fuite passe aussi par le temps) : même corps, même statut, verrou silencieux.

3. La journalisation d'auth-service est en retard sur le reste de la plateforme

`deal-service` et `message-service` émettent du `pino` structuré (`{"level":30,"name":"deal-service"}`). `auth-service` n'utilise `pino` nulle part : `user-public.controller.ts` compte **25 `console.log`** de débogage, avec emojis et données personnelles :

```
[getUserPublic] 🐼 slug=seed-aminata currentUserId=6a5c1d3b8faeca66b436c0eb
[followUser] 📁 START - slug=... userId=...
```

Ces lignes partent en clair dans les journaux de production, sans niveau, sans corrélation, et portent des identifiants de membres.

Proposition : adopter `pino` comme les autres services, et supprimer les traces de débogage (ou les passer en `debug`, désactivé par défaut).

4. Deux fichiers de 900 lignes portent l'essentiel du service

`auth.controller.ts` (900 lignes) et `user-public.controller.ts` (895) mélangent HTTP, règles métier et accès aux données. La campagne l'a montré à ses dépens : la fuite d' `ANO-API-06` était une ligne perdue au milieu de 900, et aucun contrôleur du service n'a de test — **il n'existe pas un seul `*.spec.ts` de contrôleur dans `auth-service`**. Toute la logique HTTP n'est vérifiée que par la recette manuelle.

Proposition — poursuivre le mouvement amorcé dans cette PR : sortir les règles en modules purs (`registration-rules`, `me-projection`, `session-revocation` viennent de l'être) et les tester unitairement. C'est ce découpage qui a permis d'écrire quatre tests là où il n'y en avait aucun.

5. Les rôles vivent dans le jeton, donc ils ont jusqu'à 15 minutes de retard

Constaté en jouant `API-AUTH-33` : après l'octroi du rôle `CARRIER`, `roles` reste `["SHIPPER"]` jusqu'à la reconnexion. Symétriquement, un rôle **retiré** (ou un profil admin révoqué) reste actif jusqu'à l'expiration du jeton. Or, depuis `ANO-API-07`, le middleware lit déjà l'utilisateur en base à chaque requête — `user.roles` est là, à jour, gratuitement.

Proposition — faire autorité sur les rôles lus en base plutôt que sur ceux du jeton (le jeton les garde en repli). Coût nul, cohérence immédiate, et une révocation de privilège devient aussi rapide qu'une révocation de session.

6. Deux détails à faible coût

- **`emailRegex` maison** pour valider les adresses, là où le projet a Zod partout ailleurs. Uniformiser éviterait les écarts entre ce que le contrat annonce et ce que le service accepte.
- **La fenêtre sensible est consommée par le changement de mot de passe** (constaté : il faut la rouvrir pour enchaîner un export). C'est défendable, mais ce n'est écrit nulle part — à graver dans D65 ou à documenter dans le cahier, sinon chaque testeur le redécouvrira.

Challenge expert — l'argent du membre : compte Stripe du Voyageur et finances du tableau de bord

Sujet soulevé hors recette, et point aveugle réel : la campagne `API` vérifie que l'argent est **correctement calculé** ; personne ne vérifiait qu'il est **correctement montré**. Inventaire factuel (chemins et lignes vérifiés), puis propositions classées.

A — Le parcours Stripe du Voyageur : quatre défauts qui s'enchaînent

A1. « Configurer plus tard » clôt l'onboarding sans Stripe. `completeCarrierOnboarding` (`apps/auth-service/src/controller/carrier.controller.ts:396-441`) pose `onboardingStep = COMPLETE` et `carrierStatus = ACTIVE` **sans lire un**

seul drapeau Stripe, et le front propose explicitement le bouton (`CarrierOnboardingWizard.tsx:318`, libellé « Configurer plus tard »). Constaté en recette sur API-AUTH-33 : l'étape 4 répond **200 « Carrier onboarding complete! »** avec `stripeOnboardingComplete: false`, `chargesEnabled: false`, `payoutsEnabled: false`.

Le Voyageur est donc « actif », publie ses trajets, reçoit des demandes — et découvre le blocage au moment d'accepter son premier deal, face à un Expéditeur qui attend. C'est le pire moment possible : la garde D31 (`deal-lifecycle.service.ts:151`) refuse alors avec `CARRIER_ONBOARDING_REQUIRED`.

Proposition : garder le bouton (interrompre un tunnel est légitime) mais **cesser de mentir sur l'état**. `carrierStatus` reste `ONBOARDING` tant que Stripe n'est pas opérationnel, et l'écran « Mes trajets » porte un bandeau permanent « il vous reste une étape avant de pouvoir accepter une demande ». Le bandeau existe déjà (`MyTripsList.tsx:611-617`) mais sa condition (`onboardingStep !== STRIPE/COMPLETE` et `draftCount > 0`) ne se déclenche jamais dans ce cas.

A2. Un compte qui redevient incomplet ne redescend jamais. Le webhook `account.updated` (`apps/deal-service/src/controllers/stripe-webhook.controller.ts:89-104`) écrit **exactement trois booléens** et ne touche ni `onboardingStep` ni `carrierStatus`. Si Stripe désactive un compte (pièce expirée, vérification échouée), le Voyageur reste `COMPLETE / ACTIVE`, **sans aucune notification** — l'écriture est silencieuse. Il l'apprendra par un refus d'acceptation.

Proposition : sur passage à `false`, repasser `carrierStatus` à `ONBOARDING` et **prévenir par email** — le canal existe déjà (`payout.failed` déclenche `notifyCarrierPayoutFailed`, ligne 108).

A3. Stripe dit POURQUOI, et personne ne l'écoute. `account.requirements` (`currently_due`, `past_due`, `eventually_due`, `disabled_reason`) : **zéro occurrence dans tout le dépôt**. Seuls `charges_enabled`, `payouts_enabled` et `details_submitted` sont lus (`packages/libs/payments/src/index.ts:427-432`). La plateforme sait donc dire « ça ne marche pas » mais **jamais** « il manque votre pièce d'identité ».

Proposition : stocker `requirements.currently_due` et `disabled_reason` sur `CarrierPage`, et les traduire en une phrase actionnable. C'est le seul de ces quatre points qui demande une décision de modèle (deux champs de plus), les autres sont du câblage.

A4. La garde D31 ignore `stripePayoutsEnabled`. `deal-lifecycle.service.ts:151` vérifie `stripeOnboardingComplete` et `stripeChargesEnabled` — **pas** `payoutsEnabled`. Conséquence : un deal peut être accepté, **l'argent de l'Expéditeur capturé** (ligne 180), pour un Voyageur qui ne peut pas être payé. L'argent est encaissé, la prestation faite, et le versement échouera.

Proposition : ajouter `stripePayoutsEnabled` à la garde. Une ligne, et elle empêche d'encaisser ce qu'on ne pourra pas reverser.

B — Le bouton « tableau de bord Stripe » ne fonctionne pour personne

`FinancesSection.tsx:84` calcule `stripeAccountReady = Boolean(user?.carrierPage?.stripeAccountId)`. Or `GET /auth/me` **ne projette pas `stripeAccountId`** (ni `stripePayoutsEnabled`) dans `carrierPage`. Vérifié en conditions réelles sur un Voyageur pleinement onboardé :

```
{"champs":["bio","id","name","onboardingStep","phoneE164","primaryAddress",
"stripeChargesEnabled","stripeOnboardingComplete"],
"stripeAccountId":null,"stripeOnboardingComplete":true,"stripeChargesEnabled":true}
```

Le bouton affiche donc **toujours** le toast « compte Stripe manquant », y compris pour un Voyageur irréprochable. `useUser.ts:10` type pourtant le champ comme présent : le front et le serveur ne sont pas d'accord, et rien ne l'a signalé. **Le défaut est antérieur à la campagne** (vérifié sur le commit `D65 339ecc1` : le `select` ne l'a jamais porté).

Proposition : ajouter `stripeAccountId` à la projection — ou mieux, exposer un booléen `stripeAccountReady` plutôt que l'identifiant d'un compte externe, que le front n'a aucune raison de connaître. À traiter au cahier Web, c'est une anomalie front/API.

C — Ce que le Voyageur voit de son argent, et ce qui lui manque

Ce qui existe et fonctionne : `GET /me/wallet` (`wallet.service.ts:91-107`) renvoie `upcomingCents`, `pendingCents`, `blockedCents`, `sentCents`, `sentThisMonthCents` et un historique par deal. Vérifié en recette : `{upcoming: 5500, pending: 2800, blocked: 0, sent: 6100}` avec le corridor, la contrepartie et la date pour chaque ligne. Le socle est bon.

C1. La commission n'est pas cachée par négligence, c'est un choix — mais il mérite d'être réexaminé. Le DTO Voyageur (`booking-view.mapper.ts:466-475`) est une liste blanche explicite, commentée « GAINS uniquement — ni commission ni total Shipper » (A13). Le Voyageur voit `transportCents`, jamais `totalShipperCents` ni `commissionCents`.

Il faut être précis sur le modèle : d'après le catalogue de paramètres, la commission est **calculée sur le transport et payée par l'Expéditeur**, en plus. Le Voyageur touche donc l'intégralité de `transportCents` — « ce qui reste après commission » **est** son gain affiché. Cacher la commission n'est donc pas un vol dissimulé, et l'afficher ne changerait pas son revenu.

Mais l'argument de transparence tient quand même : un Voyageur qui ignore le prix payé par l'Expéditeur ne peut pas juger si la plateforme est chère, ni vérifier qu'on ne rogne pas sa part. **Proposition** : afficher au Voyageur, sur le deal terminé, une ligne « l'Expéditeur a payé X,

dont Y de frais Yamba — votre gain : Z, intégralement ». C'est plus honnête que le silence, et cela protège Yamba de la suspicion inverse. À arbitrer, car cela touche une décision existante (A13).

C2. Le vrai manque n'est pas la commission, c'est l'écart et la date. Trois questions qu'un Voyageur se pose et auxquelles rien ne répond aujourd'hui :

- « Pourquoi mon versement est-il inférieur à mon gain annoncé ? » — `retentionCents` (retenue d'annulation tardive) peut réduire le montant, et `retentionDisposition` est exposé au Voyageur sans explication chiffrée.
- « Quand serai-je payé ? » — `payoutDueAt` est servi, mais en cas d'échec `payoutNextRetryAt` n'est jamais exposé au membre : ses seuls lecteurs sont l'admin et les crons. Le Voyageur voit « bloqué », sans horizon.
- « Pourquoi est-ce bloqué ? » — `payoutFailureReason` est lu (`wallet.controller.ts:35`) puis **réduit à deux états grossiers** (`BLOCKED` / `PENDING`, `wallet.service.ts:77-79`). Le motif n'est jamais traduit pour le membre.

C3. `blockedCents` est calculé et jamais affiché. Le contrat le documente comme « drives the banner » (`booking-wallet.schema.ts:51`), mais `WalletTab` n'affiche que `upcoming` / `sent` / `pending` (`FinancesSection.tsx:119-128`). Le bandeau réellement utilisé s'appuie sur `useMyDeals`, pas sur ce total. Travail serveur payé, jamais consommé.

C4. Aucun relevé, aucun justificatif — et c'est un sujet réglementaire. Il n'existe **aucun export côté membre** : `/admin/finances/export` et `/admin/finances/report` sont réservés à l'administration. Or un Voyageur perçoit des revenus qu'il doit déclarer, et une plateforme de mise en relation européenne relève de **DAC7** : obligation de déclarer les revenus des prestataires **et de leur en remettre une copie**. Le dossier juridique (livrable 09) mérite d'être confronté à ce point avant l'ouverture commerciale.

Proposition : un relevé annuel téléchargeable (le calcul existe déjà côté admin, il faut l'exposer au membre pour ses propres données), plus un reçu par versement.

C5. Trou d'historique. `toPayoutItem` (`wallet.service.ts:85`) renvoie `null` pour les deals `COMPLETED` antérieurs à B4 qui n'ont pas de `payoutStatus` : ils **disparaissent** du portefeuille. Un Voyageur de la première heure ne voit pas ses premiers gains. À traiter par une migration, sinon la somme affichée ne réconcilie pas avec la réalité.

D — Côté Expéditeur

Le DTO est complet (`booking-view.mapper.ts:372-393` : transport, commission, prime, total, plus `refundAmountCents` et `retentionCents`). Il manque en revanche, comme pour le Voyageur, **tout justificatif téléchargeable** : ni reçu de paiement, ni justificatif de remboursement. Pour un service payé d'avance et parfois remboursé partiellement, c'est la première réclamation attendue au support.

E — Code mort à retirer au passage

- `apps/user-ui/src/components/dashboard/sections/WalletSection.tsx` — ancienne section, **plus aucun import**.
- `apps/user-ui/src/components/carrier/OnboardingBanner.tsx` — **jamais importé** (le bandeau utilisé est celui de `MyTripsList`).
- `apps/user-ui/src/app/[locale]/dashboard/finances/preview/page.tsx` — page **mock** accessible par URL, hors navigation.

Priorisation proposée

#	Sujet	Effort	Pourquoi maintenant
1	A4 — <code>payoutsEnabled</code> dans la garde D31	une ligne	On encaisse aujourd'hui de l'argent qu'on ne pourra pas reverser
2	B — le bouton Stripe mort	une ligne de projection	Un Voyageur onboardé ne peut pas ouvrir son tableau de bord
3	A1 + A2 — l'état affiché dit la vérité, et le webhook rétrograde	petit	Le Voyageur découvre le blocage face à un client
4	C2 — date de prochain essai, motif de blocage, explication de la retenue	moyen	Ce sont les trois questions du support
5	A3 — lire <code>requirements</code> de Stripe	moyen (2 champs)	Transforme « ça ne marche pas » en « il manque ceci »
6	C4 — relevé membre	moyen	Sujet réglementaire (DAC7) avant ouverture commerciale
7	C1 — transparence de la commission au Voyageur	décision	Touche A13 : à arbitrer, pas à coder d'emblée
8	C3, C5, E — <code>blockedCents</code> , trou d'historique, code mort	petit	Dette visible, à solder en passant

F — L'onboarding est-il trop long ? Ce qu'on peut retirer sans rien perdre

Ce qui est déjà bien fait, et qu'il ne faut pas casser : le compte Stripe est créé **prérempli** (`carrier.controller.ts:238-291`) — prénom, nom, email, téléphone, date de naissance et adresse postale sont envoyés à `accounts.create`. Le Voyageur ne ressaisit donc pas ces champs chez Stripe. C'est le gros du travail d'allègement, et il est fait.

Restent trois sources de frottement, par ordre de gain.

F1. Le front exige quatre champs, le serveur en exige deux. L'écran annonce « Tous les champs sont obligatoires » et refuse de continuer sans **nom, bio, adresse et téléphone** (`CarrierOnboardingWizard.tsx:228-238`). Or `validateCarrierOnboardingData` (`auth.helper.ts:201-213`) n'exige que **phone_number et country**.

La **bio** est le champ le plus coûteux du formulaire — un texte libre, à rédiger, devant lequel on abandonne — et c'est le seul qui ne sert **ni à Stripe, ni à encaisser, ni à accepter un deal**. Elle n'alimente que la page publique.

Proposition : la rendre facultative dans le tunnel et la demander **au moment où elle sert** : à la publication du premier trajet, quand le Voyageur cherche justement à convaincre. Gain immédiat : le tunnel passe de quatre saisies à trois, dont deux triviales.

F2. Expliquer pourquoi l'adresse est demandée. Elle est nécessaire (elle préremplit Stripe et lui évite une saisie), mais rien ne le dit : l'écran demande une adresse postale à quelqu'un qui veut juste transporter un colis, ce qui ressemble à de la collecte gratuite. Une phrase — « pour ne pas avoir à la ressaisir à l'étape suivante » — coûte zéro et supprime une hésitation.

F3. Le vrai levier n'est pas le nombre de champs, c'est le MOMENT. Aujourd'hui, un membre qui veut devenir Voyageur doit fournir une pièce d'identité et un IBAN **avant d'avoir publié un seul trajet**, donc avant de savoir s'il recevra la moindre demande. C'est l'effort maximal au moment de motivation le plus faible.

Or la garde D31 ne se déclenche **qu'à l'acceptation d'un deal** (`deal-lifecycle.service.ts:151`) — le verrou a d'ailleurs déjà migré depuis la publication de trajet. Techniquement, rien n'oblige à faire Stripe si tôt.

Trois moments possibles :

Moment	Effet
Aujourd'hui — avant tout	Effort maximal, motivation minimale : c'est là qu'on perd les Voyageurs
À la première demande reçue	Motivation maximale... mais la demande expire pendant que la vérification Stripe traîne : on perd le deal ET l'Expéditeur
À la publication du premier trajet (recommandé)	Le Voyageur est engagé, il a du temps devant lui avant les demandes, et le KYC a le temps d'aboutir

Proposition : découpler. Profil minimal → publication → « avant de recevoir des demandes, finalisez votre compte de paiement ». Cela suppose que l'état affiché soit honnête (voir **A1**) : c'est la même correction, vue par l'autre bout. Sans elle, décaler Stripe ne ferait qu'aggraver la surprise du refus au premier deal.

Ce qu'il ne faut PAS raccourcir : les étapes Stripe elles-mêmes (identité, IBAN) ne sont pas négociables — elles sont imposées par la réglementation et par Stripe. Le seul levier est de les préremplir (fait) et de les demander au bon moment (à faire).

Chapitre 5.2 — trip-service (17 fiches jouées sur 18)

13 OK · 2 PARTIEL · 1 KO · 1 ➤. Les deux fiches bloquantes du chapitre passent.

Fiche	Intitulé	Gravité	Verdict	Pourquoi	Recommandation
API-TRIP-01	Recherche publique et personnalisée	majeure	OK	200 avec et sans session, enveloppe <code>{trips, nextCursor, totalCount}</code> sans success (documenté), <code>isFavorite</code> reflète bien les favoris du membre.	<code>rating</code> et <code>reviewCount</code> sont absents du JSON (le jeu d'essai n'a pas d'avis) : à confirmer au cahier Web, la carte les annonce.
API-TRIP-02	Filtres durs	bloquante	OK	Aucun trajet non publié, aucun départ passé, et une borne <code>dateFrom=2000-01-01</code> ne contourne rien. Contre-épreuve : un trajet masqué par Yamba disparaît de la recherche (2 → 1) et revient après restauration.	—
API-TRIP-03	Filtres invalides ignorés	mineure	KO	<code>categories=inventee</code> et <code>departureBuckets=matin,morning</code> sont bien tolérés (200), mais <code>mode=teleportation</code> renvoie 400 .	Voir AN0-API-10 .
API-TRIP-04	Devis et paramètres publics	majeure	OK	<code>/trips/pricing/params</code> lisible sans session (commission 12 %, plancher 8 €, kilo de référence 2 kg, prime Garantie 6 €) ; avec <code>weightKg</code> , chaque carte porte <code>transportForWeight</code> et <code>totalForWeight</code> .	—
API-TRIP-05	Facettes	majeure	OK	200 avec <code>familyCounts</code> , <code>modeCount</code> , <code>instantBookingCount</code> , <code>superTripperCount</code> , <code>profileVerifiedCount</code> , <code>verifiedTicketCount</code> , <code>totalCount</code> .	—
API-TRIP-06	Vue publique d'un trajet	majeure	OK	200, DTO public complet (lieux, dates, prix, conditions par famille, <code>isFavorite</code>).	—

Fiche	Intitulé	Gravité	Verdict	Pourquoi	Recommandation
API-TRIP-07	Favori idempotent et gardé	majeure	OK	Ajout 200 puis rejeu sans doublon (liste = 1), retrait idempotent (200 deux fois). Les deux exceptions annoncées tiennent : 404 sur un trajet inexistant, 403 OWN_TRIP sur son propre trajet.	—
API-TRIP-08	Brouillon puis publication	majeure	OK	201 DRAFT puis 200. Le point de la fiche est prouvé : <code>departureHourLocal</code> envoyé à "03:00" est recalculé à 21 , et <code>minPriceCents</code> envoyé à 1 est ignoré.	—
API-TRIP-09	Portes de publication	majeure	OK	400 avec le motif de la machine : « A complete pricing engine is required to publish... (D13) ».	—
API-TRIP-10	<code>allowedActions</code>	majeure	OK	Les actions varient réellement : un trajet portant des deals vivants n'expose ni edit ni cancel , les autres oui.	—
API-TRIP-11	Cycle de vie complet	majeure	»»	Non joué isolément ; ses transitions sont couvertes par 08 (publication), 12 (annulation refusée) et 15 (annulation, suppression).	À jouer pour compléter.
API-TRIP-12	Annulation refusée, deal vivant	bloquante	OK	409 <code>{"type":"trip","code":"TRIP_HAS_ACTIVE_DEALS","activeDeals":4}</code> — la règle D72 tient exactement.	—
API-TRIP-13	Modification, capacité immuable	majeure	PARTIEL	La modification partielle fonctionne (seul le champ envoyé est écrit). Mais la capacité est modifiable après publication (50 accepté), alors que la fiche l'annonce immuable.	Le code a raison, le cahier est trop strict. La vraie garde est plus fine et plus juste : toute modification est refusée dès qu'une réservation active existe — « Cannot edit a trip with active bookings » (vérifié : réduire la capacité à 1 kg sur un trajet à 18 kg réservés est refusé en 400). Le cas dangereux est donc impossible ; corriger l'attendu de la fiche.
API-TRIP-14	Trajet d'autrui	bloquante	OK	400 « Unauthorized. » sur lecture, modification et annulation — l'écart de sémantique annoncé par le cahier. Ce qui compte est tenu : aucune des trois ne réussit, le corps ne divulgue rien, et le prix reste à 1150.	—
API-TRIP-15	Suppression / annulation	mineure	OK	« Draft deleted. » et « Trip cancelled. », puis 404 sur les deux vues publiques.	—
API-TRIP-16	Déclarer un billet	majeure	OK	201 « 1 document(s) added. », <code>ticketVerificationStatus</code> passe à PENDING ; après retrait du document il revient à NOT_SUBMITTED .	—
API-TRIP-17	Signature de téléversement	majeure	OK	401 sans session, 200 avec (token de 36 caractères + <code>expire</code>).	—
API-TRIP-18	Suppression idempotente	mineure	PARTIEL	Le retrait d'un document répond 200, mais le rejeu répond 400 « Document not found. » au lieu d'être idempotent.	Aligner sur le favori et sur la session, qui sont idempotents : un second retrait doit répondre 200. À rejouer aussi sur <code>/uploads/:fileId</code> , que cette fiche vise en propre.

Anomalies du chapitre 5.2

ANO-API-10
Fiche : API-TRIP-03
Gravité : mineure
Appel exact : curl -s "\$BASE/trips/search?limit=3&mode=teleportation"
Attendu : 200 – les valeurs de filtre inconnues sont ignorées silencieusement
Obtenu : 400 « Invalid query parameters: mode: Invalid option: expected one of "all"|"plane"|"train"|"car" »
Reproductible : oui – et l'incohérence est nette : categories=inventee et departureBuckets=matin,morning passent en 200, seul `mode` rejette.
Impact : rupture de compatibilité. Un lien partagé, un favori de navigateur ou une ancienne version de l'application mobile portant un mode retiré du catalogue affiche une erreur au lieu d'une recherche. Les autres filtres, eux, dégradent proprement.
Piste : apps/trip-service/src/dto/trip-search.dto.ts – `mode` est un z.enum strict là où les listes passent par `csvOf`, qui filtre les valeurs inconnues. Un `.catch("all")` suffit à aligner le comportement.
CORRECTION : 08/09/2026 – `.catch("all")` posé sur les DEUX schémas (recherche et facettes), avec un test qui couvre les modes inconnus et la non-régression des filtres déjà tolérants.
CONTRE-ÉPREUVE : mode=teleportation → 200 (2 résultats) · mode=plane → 200.
ÉTAT : CLOSE

ANO-API-11
Fiche : API-TRIP-01 / 04 (révélée en jouant le tri par prix)
Gravité : majeure – masque des trajets aux Expéditeurs
Appel exact : curl -s "\$BASE/trips/search?sort=lowestPrice&limit=10" | jq '.totalCount'
curl -s "\$BASE/trips/search?sort=earliest&limit=10" | jq '.totalCount'
Attendu : le tri change l'ORDRE, pas le nombre de résultats
Obtenu : sort=earliest → 3 trajets · sort=lowestPrice → **1 trajet**, totalCount passant de 3 à 1. Mesure en base : **7 trajets** cherchables, 5 sans `comparablePriceCents` – donc exclus du tri.
Cause : le tri par prix s'appuie sur `comparablePriceCents` (D33) et EXCLUT les trajets qui ne l'ont pas (`where.comparablePriceCents = { not: null }`, trip-search.controller.ts:259). Le code de création le calcule correctement – le trajet créé pendant la recette l'avait (2400 = 1200 × 2 kg). Ce sont les trajets ANTÉRIEURS à D33 qui ne l'ont pas.
Reproductible : oui
Impact : un Expéditeur qui trie par prix – le tri le plus utilisé d'une place de marché – ne voit qu'une fraction de l'offre, et `totalCount` le lui cache. Les Voyageurs concernés sont invisibles sans le savoir.
Correction : le script `packages/libs/prisma/scripts/backfill-comparable-price.ts` EXISTE (il était prévu par D33). Joué sur la base de recette : « 37 trips lus, 8 mis à jour ». Après quoi sort=lowestPrice renvoie bien 3 trajets, ordonnés 9,50 → 11,50 → 12,00 €.
À TRANCHER : ce script a-t-il été joué sur la base de PRODUCTION ? Si non, la même amputation du tri par prix y est active aujourd'hui.
Reste à faire : (a) le seed ne pose PAS `comparablePriceCents` – toute recette du tri par prix est donc faussée tant qu'on ne rejoue pas le backfill après un seed ; (b) sur le fond, exclusion silencieusement est discutable : un trajet sans prix comparable devrait être rangé en fin de liste, pas retiré, et `totalCount` ne devrait jamais varier selon le tri.
CORRECTION : 08/09/2026 – la cause est traitée, pas le symptôme, sur trois plans :
(1) INVARIANT à la publication (trip.controller.ts) : les champs dénormalisés sont recalculés au moment où le trajet devient visible. Un trajet d'avant D33 se REPARE donc tout seul en étant publié – vérifié : champ effacé à la main, null avant publication, 1800 après.
(2) L'EXCLUSION EST RETIRÉE du tri (trip-search.controller.ts) : un tri change l'ORDRE, jamais le NOMBRE. Tri secondaire sur `minPriceCents` puis `id` pour rester déterministe si une valeur manquait malgré tout – ces trajets remontent en tête plutôt que de disparaître, car un défaut d'affichage vaut mieux qu'une offre invisible.
(3) Le SEED pose désormais le champ : sans cela, toute recette du tri par prix reste faussée, et c'est précisément ce qui avait masqué le défaut.
CONTRE-ÉPREUVE : base volontairement privée du champ sur les 28 trajets publiés – sort=earliest 2, sort=lowestPrice 2, sort=bestRated 2. Le tri ne perd plus rien. Avant correction, lowestPrice aurait renvoyé 0. Backfill rejoué ensuite (40 lus, 28 mis à jour).
PRODUCTION : sans objet – rien n'est en production à ce jour.
ÉTAT : CLOSE

Écarts du cahier (chapitre 5.2)

10. **API-TRIP-13** annonce la capacité « immuable après publication » : elle ne l'est pas, et la garde réelle (aucune modification dès qu'une réservation existe) est meilleure.
11. **API-TRIP-01** liste `rating` et `reviewCount` parmi les champs d'une carte : ils n'apparaissent pas sur un jeu d'essai sans avis.

Chapitre 5.3 — deal-service, le cœur transactionnel (18 fiches jouées sur 23)

13 OK · 2 PARTIEL · 3 KO. Les cinq fiches bloquantes jouées passent, sauf celles portant les trois anomalies ci-dessous — toutes corrigées dans la campagne.

Fiche	Intitulé	Gravité	Verdict	Pourquoi
API-DEAL-01	Autoriser un devis	majeure	OK	409 <code>QUOTE_DIVERGENCE</code> avec les deux montants, puis 201 <code>provider: FAKE</code> , <code>clientSecret: null</code> .
API-DEAL-02	Catalogue des refus typés	bloquante	KO	Six codes vérifiés — <code>QUOTE_DIVERGENCE</code> , <code>CAPACITY_EXCEEDED</code> , <code>FAMILY_REFUSED</code> , <code>OWN_TRIP</code> , <code>TRIP_NOT_BOOKABLE</code> répondent tous 409 <code>type:"booking"</code> . Le septième, <code>NEW_ACCOUNT_CAP</code> , ne se déclenchait jamais → <code>ANO-API-12</code> .
API-DEAL-05	Instantané de prix immuable	bloquante	OK	Prix du trajet multiplié par 9 : le devis du deal ne bouge pas d'un centime . La règle non négociable tient.
API-DEAL-06	Vues à liste blanche	bloquante	PARTIEL	Le point critique passe : le code de livraison est invisible au Voyageur (<code>fuites: []</code>), qui ne voit que ses gains (<code>transportCents</code>), jamais les totaux Expéditeur. Mais la vue Voyageur porte <code>recipient complet</code> (nom, téléphone, email) dès <code>ACCEPTED</code> .
API-DEAL-07	Acceptation et capture	majeure	OK	Charte refusée → 400 ; acceptation → 200, <code>capturedAt</code> posée, et la conversation du deal s'ouvre (<code>canWrite: true</code>).
API-DEAL-08	Refus de l'acceptation	majeure	OK	Rejeu → 409, deal terminal → 409, Expéditeur qui accepte → 403 « Only the carrier can accept this deal ».
API-DEAL-09	Le Voyageur refuse	majeure	OK	Le motif est une liste fermée (refus champ par champ avec les valeurs admises) ; <code>DECLINED</code> + remboursement intégral (2834 = le total).
API-DEAL-10	Retenue annoncée = retenue appliquée	majeure	OK	Aperçu <code>{refundCents: 2834, retentionCents: 0, fullRefundUntil}</code> → annulation <code>refundAmountCents: 2834</code> . L'annonce et l'effet coïncident.
API-DEAL-11	Cinq points d'inspection	bloquante	OK	Inspection partielle → 400 ; sans photo → 400 ; complète → 200 <code>PICKED_UP</code> .
API-DEAL-12	Le code n'est pas dans la réponse	bloquante	OK	Zéro séquence de six chiffres, et exactement quatre champs : <code>bookingId</code> , <code>status</code> , <code>refundAmountCents</code> , <code>currencyCode</code> .
API-DEAL-15	Le verrou du code	bloquante	PARTIEL	Compteur 2 → 1 → 0 avec <code>DELIVERY_CODE_INVALID</code> , puis <code>DELIVERY_LOCKED</code> , et le bon code est refusé pendant le verrou . Deux réserves : <code>lockUntilSeconds</code> est <code>null</code> (le Voyageur ignore combien de temps attendre) et le code passe ensuite à <code>TRANSITION_NOT_ALLOWED</code> , donc un client qui traduit <code>DELIVERY_LOCKED</code> perd le fil.
API-DEAL-16	Régénérer le code	majeure	OK	Cinq régénérations acceptées, codes tous différents (CSPRNG), la sixième refusée en 409 <code>CODE_REGENERATION_LIMIT</code> .
API-DEAL-17	La confirmation lance le versement	majeure	OK	<code>COMPLETED</code> , <code>payoutStatus: "SENT"</code> , <code>payoutAmountCents: 5500</code> — et le portefeuille du Voyageur passe de 6 100 à 11 600 . La chaîne argent complète fonctionne.
API-DEAL-18	Le litige gèle le versement	majeure	OK	Description < 50 caractères → 400 ; engagement d'honnêteté obligatoire → 400 ; litige complet → <code>DISPUTED</code> et <code>payoutStatus: "FROZEN"</code> .
API-DEAL-20	Notation en double aveugle	majeure	KO	Le double aveugle lui-même est correct (<code>myRating</code> visible, <code>counterpartRating: null</code> , <code>revealedAt: null</code>). Mais noter avec des critères était impossible → <code>ANO-API-14</code> .
API-DEAL-21	Un seul lien par deal	mineure	OK	Le rejeu rend le même jeton ; le Voyageur reçoit 403 « Only the shipper shares the tracking link ».

Fiche	Intitulé	Gravité	Verdict	Pourquoi
API-DEAL-22	Page destinataire publique	bloquante	KO	Répondait 500 → ANO-API-13. Une fois corrigée, elle est exemplaire : prénoms seuls, aucun code, aucun montant , aucun email, aucun nom de famille, aucun téléphone ; jeton inventé → 404.
API-DEAL-23	Listes bornées au propriétaire	majeure	OK	me/deals, me/notifications, me/wallet : 401 sans session, 200 avec.

Anomalies du chapitre 5.3

ANO-API-12

Fiche : API-DEAL-02 · Gravité : majeure · ÉTAT : CLOSE (PR #239)
 Constat : un compte créé 30 secondes plus tôt obtenait une intention de paiement pour un colis déclaré à 5 000 €. Le plafond CNF-06 existe et fonctionne – mais à la CRÉATION du deal, une fois la carte déjà pré-autorisée.
 Cause : deal-request.service.ts passait `declaredValueCents: 0` EN DUR à `assertWithinCaps`, sur la ligne dont le commentaire dit « avant d'autoriser l'argent ».
 Correction : `CreatePaymentIntentRequest` accepte la valeur déclarée (FACULTATIVE), qui est confrontée aux plafonds avant l'autorisation.
 Contre-épreuve : compte neuf + 5000 € → 409 dès l'autorisation · sans valeur → 201 inchangé · 200 € → 201 · compte ancien + 5000 € → 201.

ANO-API-13

Fiche : API-DEAL-22 · Gravité : BLOQUANTE · ÉTAT : CLOSE (PR #240)
 Constat : GET /api/track/{token} – la page que l'Expéditeur partage au destinataire – répondait 500. « Unknown field `cancelledAt` for select on model `Booking` ».
 Cause : `Booking` n'a pas de `cancelledAt` (seul `cancelReason` existe). C'est le DEUXIÈME cas identique de la campagne après ANO-API-09 (export RGPD), et pour la même raison : les specs injectent un faux Prisma, qui ne valide aucun nom de champ.
 Correction : champ retiré du select – et surtout, le garde-fou devient SYSTÉMIQUE : prisma-select-fields.spec.ts lit tous les fichiers de deal-service, extrait chaque `select` et le confronte à prisma/schema.prisma.
 Contre-épreuve : page à 200 ; contenu vérifié minimal ; jeton inventé → 404 ; test vérifié en réintroduisant le bug.

ANO-API-14

Fiche : API-DEAL-20 · Gravité : majeure · ÉTAT : CLOSE
 Constat : noter AVEC des critères était impossible. Un Expéditeur notant un Voyageur avec ses trois critères (PUNCTUALITY, COMMUNICATION, PARCEL_CARE) recevait 400 en réclamant DECLARATION_CLARITY et RESPONSIVENESS – les critères de l'AUTRE rôle. Seul le contournement « ne pas envoyer de critères » marchait.
 Cause : `z.record(RatingCriterionSchema, RatingVoteSchema)`. Depuis **Zod 4**, un `record` dont la clé est un enum est EXHAUSTIF : toutes les valeurs de l'énumération deviennent obligatoires. Preuve isolée :
 `z.record(z.enum(['A','B']), z.string()).safeParse({A:'x'})` → échec ;
 `z.partialRecord(...)` → succès. Le piège est silencieux : le typecheck passe, et les tests qui n'envoient pas de critères passent aussi.
 Correction : `z.partialRecord` sur les deux occurrences (requête et réponse). Vérifié qu'aucun autre `z.record(<enum>, ...)` n'existe dans le dépôt – les autres records ont des clés `string`, non concernées.
 Contre-épreuve : trois critères du rôle → la validation passe (409 « déjà noté », plus 400) ; critère inconnu → toujours 400 ; double aveugle intact.

À arbitrer (le code contredit le cahier et s'en explique)

1. **API-DEAL-06** — la vue Voyageur porte `recipient` complet dès `ACCEPTED`. Le code l'assume : « Le destinataire est visible côté Carrier : il en a besoin pour livrer. » C'est vrai du **nom**. Ça ne l'est pas de l'**email**, qui ne sert à rien pour livrer, et le **téléphone** gagnerait à n'apparaître qu'à `PICKED_UP`. Le destinataire est un **tiers** qui n'a rien signé : c'est un sujet de minimisation, à trancher avec le dossier 11.
2. **API-DEAL-15** — le verrou du code de livraison ne dit pas **quand réessayer** (`lockUntilSeconds: null`), et le code métier change entre le troisième essai et les suivants. Même thème que côté auth : un refus doit donner un horizon.

Chapitre 5.4 — message-service (11 fiches sur 12)

11 OK · 1 ➡. Aucune anomalie. C'est le chapitre le plus propre de la campagne.

Fiche	Intitulé	Gravité	Verdict	Pourquoi
API-MSG-01	Fil créé au premier accès	majeure	OK	200, identifiant créé, access: {canRead:true, canWrite:true}.
API-MSG-02	Aucun fil pour un tiers	bloquante	OK	Tiers → 403 « You are not a party to this deal » ; sans session → 401.
API-MSG-03	Les deux gardes du message	bloquante	OK	Message ordinaire → 201 flaggedContact:false · le code de livraison → 400 DELIVERY_CODE_IN_MESSAGE · un numéro de téléphone → 201 avec flaggedContact:true · texte vide → 400. La distinction voulue par D61 est exacte : le code se refuse, les coordonnées se marquent.
API-MSG-04	Fenêtre d'écriture et motif	majeure	OK	Litige → canWrite:false, reason:"DISPUTE_OPEN" ; deals terminés → writeClosesAt à 14 jours. Le serveur tient la règle : écrire dans un fil en litige → 400 « This conversation is read-only (DISPUTE_OPEN) ».
API-MSG-05	Marquer lu	mineure	OK	Compteur de non-lus 5 → 0 après POST /read, readAt retourné.
API-MSG-06	Le rendez-vous est un objet	majeure	OK	201, status:"PROPOSED", kind:"PICKUP" — un objet à part entière, pas un fil de messages.
API-MSG-07	Contre-proposition	majeure	OK	La nouvelle proposition passe la précédente en CANCELLED et l'historique reste visible — la traçabilité prime sur la propreté de la liste.
API-MSG-08	On n'accepte que la proposition de l'autre	majeure	OK	Sa propre proposition → 400 OWN_PROPOSAL ; acceptée par l'autre partie → ACCEPTED.
API-MSG-09	Le numéro ne s'ouvre pas avant l'heure	majeure	OK	Rendez-vous le 20/09 à 17 h, demande le 08/09 → 400 TOO_EARLY avec l'heure exacte d'ouverture (15 h UTC = 17 h - 2 h). La règle D61 tient au caractère près.
API-MSG-10	Signaler un message	majeure	OK	Message de l'autre → signalement créé, rejeu → 409 « already reported » ; son propre message → 400 ; motif hors liste → 400.
API-MSG-11	Réponses rapides	mineure	OK	Couvert par API-GW-10 : clé stable, texte selon la langue du lecteur (preferredLocale).
API-MSG-12	Lecture admin journalisée	majeure	⚡	Suppose une session administrateur avec 2FA TOTP, hors de portée d'une campagne API. À jouer au cahier Admin.

Une remarque transverse, mineure

Trois refus de ce service portent leur code **dans le message** plutôt que dans details.code : OWN_PROPOSAL, DISPUTE_OPEN, et le motif du verrou d'écriture — par exemple « This conversation is read-only (DISPUTE_OPEN). ». Le cahier demande de juger sur le **code**, pas sur le message anglais : un client doit pouvoir traduire sans analyser une phrase. Deux lignes par site suffiraient à les exposer dans details. Rien d'urgent, mais c'est le même thème que les verrous qui ne donnent pas leur horizon.

Chapitre 5.5 — notification-service (4 fiches)

3 OK · 1 KO — mais ce KO a révélé **deux anomalies**, dont une bloquante, et l'une d'elles rendait muet tout l'envoi d'emails transactionnels.

| API-NOTIF-01 | Lire ses notifications | mineure | **OK** | 200, unreadCount juste, types cohérents avec les clés d'événement. | | API-NOTIF-02 | Marquer lu | mineure | **OK** | Idempotent des deux côtés : updatedCount **7** puis **0** au rejeu, unreadCount final **0**. | | API-NOTIF-03 | **La notification d'autrui** | **bloquante** | **OK** | **403** (pas destinataire) · **404** (inexistante) · **400** (identifiant malformé) · **401** (sans session). Les quatre statuts sont **distincts et corrects** — le tableau complet de la sémantique, mieux tenu qu'ailleurs dans la plateforme. La notification de l'autre membre reste non lue. | | API-NOTIF-04 | Une transition réelle produit une notification | majeure | **KO** → **corrigé** | Le compteur du Voyageur passe bien de 14 à 15 et booking.requested arrive en tête : la chaîne **transition** → **outbox (même transaction)** → **Redpanda** → **consommateur** → **notification** fonctionne. Mais la fiche a buté sur deux défauts, voir ci-dessous. |

Anomalies du chapitre 5.5

ANO-API-16	
Fiche	: API-NOTIF-01 / 04 · Gravité : BLOQUANTE · ÉTAT : CLOSE
Constat	: GET /me/notifications répondait 500 pour le Voyageur – et pour tout membre ayant reçu un message. La boîte entière devenait inaccessible.
Cause	: le type de notification était une union FERMÉE de clés `booking.*` et d'une clé système. Or message-service produit aussi `conversation.message_posted` et `conversation.meetup_proposed` (D61) : la validation de la réponse échouait sur ces lignes, et une seule suffisait à faire tomber la liste. Vérifié en base : 5 notifications `conversation.message_posted` et 3 `conversation.meetup_proposed` existaient bel et bien.
Correction	: l'union reste FERMÉE – le mapper est un garde, pas un tuyau, et un type inventé (`booking.hacked`) doit continuer d'être refusé, ce qu'un test existant exigeait à juste titre – mais elle est COMPLÈTE : elle réutilise la liste que message-service publie déjà, plutôt que d'en recopier des clés qui divergeraient au prochain événement. Et la robustesse est traitée là où elle doit l'être : à la lecture, une notification illisible est ignorée et JOURNALISÉE (type + identifiant), au lieu d'emporter la boîte entière. Un membre garde ses autres notifications, l'exploitation voit ce qu'il faut ajouter au contrat.
Contre-épreuve	: la boîte du Voyageur répond 200 avec ses 14 notifications, dont les `conversation.*` qui la faisaient tomber. Les 99 tests du service passent, y compris celui du rejet strict.

ANO-API-17	
Fiche	: API-NOTIF-04 · Gravité : majeure · ÉTAT : CLOSE
Constat	: AUCUN email transactionnel de deal ne partait. Chaque envoi échouait en ENOENT et l'événement était marqué « Booking email send failed – marked FAILED » – sans que rien ne remonte au membre ni à l'appelant.
Cause	: le dossier des gabarits EJS était résolu depuis `process.cwd()` : `path.join(process.cwd(), "apps/notification-service/src/emails/templates")`. Le chemin n'est donc juste QUE si le service est lancé depuis la racine du dépôt. Lancé depuis son propre dossier – ce que fait `scripts/smoke-services.sh`, et ce que fera n'importe quel conteneur – il se doublait : `../apps/notification-service/apps/notification-service/src/emails/templates/...`
Correction	: les gabarits sont copiés dans le bundle (webpack `assets`) et résolus depuis `__dirname`, avec un repli sur les sources pour les tests unitaires. Le chemin ne dépend plus du répertoire de lancement.
Contre-épreuve	: après création d'un deal, DEUX emails sont réellement arrivés dans Mailpit – « Nouvelle demande de transport Paris → Brazzaville » au Voyageur et « Reçu : paiement autorisé pour ton envoi » à l' Expéditrice . Avant la correction, les deux étaient marqués FAILED.
Portée	: ce défaut ne se voyait ni au typecheck, ni aux tests unitaires, ni au smoke (qui ne vérifie que /health). Il fallait provoquer une transition réelle et aller regarder la boîte aux lettres – c'est précisément ce que la bascule Mailpit du début de campagne a rendu possible.

Chapitre 6 — Sécurité (12 fiches jouées sur 15)

11 OK · 1 KO · 3 (deux webhooks renvoyés au chapitre 8, une fiche admin au cahier Admin).

Le chapitre confirme l'essentiel : jetons, cloisonnement des rôles, sanctions, secrets, traces. La seule anomalie est une **fuite par le temps sur la connexion**, la plus exposée des surfaces.

Fiche	Intitulé	Verdict	Pourquoi
API-SEC-01	401 partout sans jeton	OK	Neuf routes protégées, neuf 401.
API-SEC-02	Jeton expiré, invalide, tronqué	OK	Forgé → 401, tronqué → 401, alg=none → 401 (l'attaque classique ne passe pas), témoin valide → 200.
API-SEC-04	Une session membre n'ouvre rien d'admin	OK	Six routes d'administration, aucune ouverte — alors même que ce compte porte le profil SUPPORT. Les cookies admin sont bien une session à part.
API-SEC-06	Compte restreint, compte suspendu	OK	RESTRICTED : lecture 200, écriture 403 avec un message clair. SUSPENDED : la session déjà ouverte tombe immédiatement (401 ACCOUNT_SUSPENDED) et la reconnexion est refusée — la sanction agit par lecture à chaque requête, pas par une révocation différée.

Fiche	Intitulé	Verdict	Pourquoi
API-SEC-08	Balayage du code de livraison	OK	Six surfaces (2 à 32 Ko de corps) sans le code — deal Voyageur, listes, notifications, fil, portefeuille, page publique — et la vue Expéditrice avec . C'est ce témoin positif qui rend le test valable.
API-SEC-09	Le destinataire n'est pas exposé	OK	Page publique : recipientFirstName seul. Vue Voyageur : nom et téléphone, plus d'email — l'effet de ANO-API-15.
API-SEC-10	Aucun secret technique	OK	Six réponses balayées (passwordHash, totpSecret*, deliveryCodeHash, deliveryCodeEncrypted, stripeAccountId, clés) : aucune occurrence.
API-SEC-11	Aucune trace de pile	OK	Quatre erreurs provoquées : ni at Object, ni chemin node_modules, ni mention de Prisma.
API-SEC-14	Aucune énumération possible	KO → corrigé	Corps identiques, mais 168,6 ms contre 20,4 ms à la connexion. Voir ANO-API-18.
API-SEC-15	CORS	OK	Les trois origines déclarées reçoivent l'en-tête ; les autres sont refusées. Réserve mineure : le refus prend la forme d'un 500 , alors qu'il n'a rien d'une erreur serveur — sans conséquence de sécurité (le navigateur bloque de toute façon), mais trompeur dans les journaux et pour qui déboguera un nouveau front.

Anomalie du chapitre 6

ANO-API-18	
Fiche	: API-SEC-14 · Gravité : BLOQUANTE · ÉTAT : CLOSE
Appel exact	: for i in \$(seq 1 20); do curl -s -o /dev/null -w '%{time_total}\n' \ \ -X POST "\$BASE/auth/login" -H 'Content-Type: application/json' \ -d '{"email":"<adresse>","password":"MauvaisMotDePasse-1!"}'; done sort -n
Attendu	: rien ne distingue un compte existant d'un compte inconnu
Obtenu	: corps et statut identiques (401 « Invalid email or password ») — mais \ · compte existant médiane 168,6 ms (min 146,7 / max 370,9) \ · compte inexistant médiane 20,4 ms (min 14,1 / max 31,1) \ Distributions DISJOINTES : le minimum du cas « existe » dépasse de loin le maximum du cas « n'existe pas ». Un seul appel suffit à trancher.
Cause	: le cas d'école. `if (!user) return next(...)` est placé AVANT `bcrypt.compare` : un compte inexistant ne paie jamais le coût du hachage, un compte existant le paie toujours.
Impact	: énumération de comptes sur la surface la PLUS exposée de la plateforme. C'est le point de départ du bourrage d'identifiants — et cela annule le soin pris (à juste titre) à rendre les corps rigoureusement identiques. La même faille existait sur la connexion ADMIN, où l'enjeu est plus grand encore : elle laissait deviner QUI est administrateur.
Correction	: apps/auth-service/src/utls/password-timing.ts — le mot de passe est comparé dans TOUS les cas, contre un hachage leurre de même coût (bcrypt, 10 tours) calculé au chargement du module, jamais écrit en dur. Appliqué à la connexion membre ET à la connexion admin.
Contre-épreuve	: compte existant 179,0 ms (157,8 / 448,7) · compte inexistant 180,6 ms (157,4 / 507,8) — distributions superposées, l'écart a disparu. Connexion valide toujours 200. Tests : password-timing.spec.ts (6 cas, dont un qui refuse un rapport de durée supérieur à 3 entre les deux chemins).

Ce que ce chapitre m'a appris sur mes propres tests

Le balayage du code de livraison (API-SEC-08) a été écrit **trois fois**. Les deux premières versions affichaient « code absent partout » — un résultat rassurant et **entièrement faux** : la première envoyait les cookies dans un seul argument mal formé (toutes les réponses étaient des 401), la seconde utilisait `head -n -1`, qui n'existe pas sur macOS (tous les corps étaient vides).

Un test de sécurité qui ne trouve rien doit être **suspecté avant d'être cru**. Ce qui a sauvé celui-ci, c'est le **témoin positif** : exiger que le code soit **présent** là où il est légitime. Sans cette ligne, l'erreur passait pour un succès.

Chapitre 7 — Idempotence et concurrence (10 fiches sur 10)

Ce chapitre pose une seule question, sous dix formes : **que se passe-t-il quand le même geste arrive deux fois ?** Deux clics, deux onglets, un réseau qui bégaye, deux Expéditeurs qui visent les mêmes kilos. Un rejeu n'est pas un cas limite : c'est le comportement normal d'un client mobile.

Fiche	Ce qui est éprouvé	Résultat	Pourquoi	Recommandation
API-IDEM-01	Une intention de paiement ne sert qu'une fois	OK	409 PAYMENT_ALREADY_USED , un seul deal	—
API-IDEM-02	Rejouer une acceptation	OK	200 puis 409 ; acceptedAt identique, une seule capture, un seul booking.accepted	—
API-IDEM-03	Rejouer une remise	OK	200 puis 409 ; deliveredAt et payoutDueAt figés ; le rejeu ne consomme aucune tentative de code	—
API-IDEM-04	Les gestes déclarés idempotents	OK	favori, abonnement, tout-lu, lecture de fil : même code deux fois, aucun doublon ; le lien de suivi rend le même jeton	—
API-IDEM-05	Rejouer un événement consommé	OK	même webhook signé deux fois : 200/200, suppressed vrai puis faux ; un type inconnu répond 200 ignored	—
API-IDEM-06	Deux acceptations simultanées	OK	un 200, deux 409 PAYMENT_STATE_CONFLICT , une seule capture	—
API-IDEM-07	Deux réservations sur les derniers kilos	KO	la capacité reste juste, mais le perdant recevait 500	ANO-API-19
API-IDEM-08	Deux acceptations de rendez-vous	KO	200 + 400 corrects, mais le refus partait sans details.code	ANO-API-20
API-IDEM-09	Deux régénérations de code	KO	le compteur ne descend que d'un, mais le perdant recevait 500	ANO-API-21
API-IDEM-10	Deux rafraîchissements de session	OK	200 + 401, une seule session vivante ensuite	—

Ce que le chapitre démontre, et qui est rassurant : les gardes métier tiennent toutes. Aucun double débit, aucun double décrétement de capacité, aucun compteur faussé, aucune date de jalon déplacée par un rejeu. Le mécanisme est partout le même — une **écriture conditionnelle** (statut attendu, compteur attendu) plutôt qu'une lecture suivie d'une écriture — et il fait exactement ce qu'on attend de lui.

Ce que le chapitre révèle, et qui l'était moins : quand deux transactions Mongo se disputent le même document, la base rejette la perdante avec le code P2034 (« *write conflict ... please retry* »). Personne ne rattrapait ce cas : il traversait la pile et sortait en **500** « **Something went wrong** ». Deux fiches sur dix sont tombées dessus, sur deux chemins différents — donc ce n'était pas un accident local, mais un **trou de famille**.

Anomalies du chapitre 7

ANO-API-19

Fiche : API-IDEM-07 · Gravité : BLOQUANTE · ÉTAT : CLOSE

Appel exact : deux POST /api/deals simultanés (deux Expéditeurs, 4 kg chacun) sur un trajet où il ne reste que 5 kg

Attendu : un 201, un 409 CAPACITY_EXCEEDED

Obtenu : un 201 — et un 500 {"status":"error","error":"Something went wrong, please try again!"}. La capacité, elle, restait JUSTE (17 → 5 kg, une seule réservation comptée) : la garde métier a tenu.

Cause : MongoDB rejette la transaction perdante avec « Transaction failed due to a write conflict or a deadlock. Please retry your transaction » (Prisma P2034), levée dans le \$transaction de createBooking. Ce n'est pas une décision métier : c'est un accident d'infrastructure, et la base dit elle-même quoi en faire — réessayer.

Impact : sur une place de marché, la course aux derniers kilos est la situation NORMALE, pas le cas limite. L'Expéditeur perdant voyait une panne au lieu d'un refus compréhensible, ne savait pas s'il devait recommencer, et le 500 partait dans Sentry comme une vraie erreur serveur.

Correction : apps/deal-service/src/lib/write-conflict-retry.ts — withWriteConflictRetry rejoue l'opération UNIQUEMENT sur P2034 (3 tentatives, délai court et légèrement aléatoire pour ne pas remettre les perdants au coude à coude). Toute autre erreur remonte intacte : réessayer ce qu'on ne comprend pas est le meilleur moyen de doubler un effet de bord. Après ANO-API-21, la protection a été posée sur les CINQ transactions de deal-service, dont applyBookingTransition — le writer central de toutes les transitions.

Contre-épreuve : mêmes deux appels simultanés → A:201, B:409 CAPACITY_EXCEEDED, capacité 5 → 1 kg. Tests : write-conflict-retry.spec.ts (4 cas).

ANO-API-20	
Fiche	: API-IDEM-08 · Gravité : MAJEURE · ÉTAT : CLOSE
Appel exact	: deux POST /api/messages/conversations/{id}/meetups/{id}/accept simultanés
Attendu	: 200 + 400, un seul acceptedAt – et un refus exploitable par le client
Obtenu	: 200 + 400 avec un seul acceptedAt (la garde optimiste fait son travail), mais le 400 portait {"message":"This meeting was just changed. Reload the conversation.", "details":null}. Aucun code.
Cause	: message-service mélangeait deux conventions. Certains refus portaient déjà un details.code (DELIVERY_CODE_IN_MESSAGE, fenêtre du téléphone), d'autres collaient la raison machine DANS la phrase anglaise : « This conversation is read-only (DISPUTE_OPEN). ». Lisible par un humain anglophone, inexploitable par un programme. Et deux des quatre classes d'erreur – ForbiddenError et NotFoundError – n'acceptaient même pas de `details` : un 403 ou un 404 métier ne POUVAIT pas porter de code.
Impact	: le front ne peut ni traduire le refus, ni le distinguer d'une saisie invalide, ni décider s'il faut recharger le fil. C'est exactement la règle non négociable « un refus métier porte un details.code, et ce code atteint le client » – non tenue sur tout un service.
Correction	: packages/error-handler/index.ts – ForbiddenError et NotFoundError acceptent désormais `details` (additif, aucun appelant existant cassé). apps/message-service : les 16 refus portent un code (CONVERSATION_READ_ONLY + reason, MEETUP_CHANGED, MEETUP_NOT_ACCEPTABLE + reason, INVALID_MEETUP_SLOT + reason, EMPTY_MESSAGE, NOT_A_PARTY, CONVERSATION_NOT_OPEN, MEETUP_NOT_FOUND, REPORT_ALREADY_REVIEWED...), et la raison machine a quitté la phrase. Garde-fou : refusal-codes.spec.ts LIT les sources du service et refuse tout jet d'erreur métier sans code, ainsi que toute raison cachée entre parenthèses dans le message – même famille que les tests qui lisent prisma/schema.prisma (ANO-API-09, ANO-API-13).
Contre-épreuve	: rendez-vous déjà accepté → 400 {"code":"MEETUP_NOT_ACCEPTABLE", "reason":"NOT_PROPOSED"} · écriture pendant un litige → 400 {"code":"CONVERSATION_READ_ONLY", "reason":"DISPUTE_OPEN"} · non-partie au deal → 403 {"code":"NOT_A_PARTY"}.

ANO-API-21	
Fiche	: API-IDEM-09 · Gravité : MAJEURE · ÉTAT : CLOSE
Appel exact	: deux POST /api/deals/{id}/code/regenerate simultanés
Attendu	: un succès, un refus métier, compteur décrémenté d'une seule unité
Obtenu	: un 200 (compteur 5 → 4, donc juste) et un 500. Journal : P2034 dans regenerateCode – la MÊME cause qu'ANO-API-19, sur un autre chemin.
Cause	: la protection posée pour ANO-API-19 avait été appliquée à l'endroit où le défaut avait été observé (la création de deal), pas à la famille. Or toutes les transitions passent par applyBookingTransition : accepter, refuser, remettre, annuler, régénérer un code... Chacune pouvait rendre un 500 le jour où deux appels se croisent.
Impact	: un membre qui double-clique sur « régénérer le code » voit une panne. Plus largement : n'importe quelle transition concurrente pouvait le faire.
Correction	: withWriteConflictRetry appliqué au WRITER CENTRAL (booking-write.ts, applyBookingTransition) et aux transactions restantes de deal-service (deal-mediation, admin-finance ×2). Aucune ne fait d'appel externe en son sein – le fournisseur de paiement est toujours sollicité AVANT la transaction – donc un rejeu est sans effet de bord.
Contre-épreuve	: deux régénérations simultanées → 200 + 409 TRANSITION_NOT_ALLOWED, compteur 4 → 3 (une seule unité).

La leçon transverse du chapitre

Corriger là où le défaut a été **vu** n'est pas corriger le défaut. ANO-API-19 avait été refermée sur la création de deal ; la même cause a resurgi deux fiches plus loin sur la régénération de code. Le bon geste était de remonter au **writer commun** — `applyBookingTransition` — et d'y poser la protection une fois pour toutes. Le même raisonnement avait déjà servi pour ANO-API-09 → ANO-API-13 (un test qui lit le schéma pour UN fichier, puis pour TOUT le service).

C'est aussi ce qui justifie les deux garde-fous de ce chapitre : `write-conflict-retry.spec.ts` et `refusal-codes.spec.ts` ne testent pas un comportement, ils testent une **règle du code** — et une règle tenue par un test ne se redéfait pas au prochain ajout.

Chapitre 8 — Webhooks (8 fiches sur 8)

Deux flux entrants, deux mécanismes de signature, deux chemins d'accès. Ce sont les **seuls points d'entrée que la plateforme expose au monde extérieur** sans session : leur seule défense est la signature.

Les trois fiches Stripe ont d'abord été jouées avec des événements **signés à la main**, la CLI n'étant pas installée sur le poste ; elles ont été **rejouées le soir même avec la vraie CLI** (§ « Rejeu avec la CLI Stripe » plus bas), ce qui lève la première réserve du verdict.

Fiche	Ce qui est éprouvé	Résultat	Pourquoi	Recommandation
API-HOOK-01	Écoute Stripe locale	OK (rejouée le soir même)	CLI installée et lancée avec la clé de test ; secret de signature affiché, chaque événement transmis journalisé avec le code rendu	—
API-HOOK-02	Les quatre réponses Stripe	OK	200 (type non traité) · 400 (en-tête absent) · 400 (signature invalide) · 501 (sans secret)	—
API-HOOK-03	Les événements traités	OK	annulation par autorisation morte, accusé de capacité, drapeaux du Voyageur suivis dans les deux sens	—
API-HOOK-04	Rejeu Stripe	OK	200/200 et un seul <code>booking.cancelled</code>	—
API-HOOK-05	Les trois réponses email	OK	503 sans secret, 401 signature absente/fausse, 200 valide	—
API-HOOK-06	Événement email signé	OK	la trace réelle passe SENT → DELIVERED	—
API-HOOK-07	Rebond dur → suppression	OK	suppression posée, et plus aucun email ne part vers cette adresse	—
API-HOOK-08	Type inconnu / rejeu	OK	200 <code>ignored</code> ; rejeu sans second effet	—

Aucune anomalie. C'est le premier chapitre de la campagne qui se termine sans écart, et ce n'est pas un hasard : les deux webhooks reposent sur des **règles pures déjà testées** (`verifySvixSignature` sur vecteurs, `interpretEmailEvent` type par type, `constructStripeWebhookEvent` côté Stripe), et le contrôleur au-dessus se contente d'orchestrer.

La preuve qui compte le plus, et comment elle a été obtenue

La fiche API-HOOK-07 ne se contente pas de vérifier que le rebond dur pose `HARD_BOUNCE` : elle demande la **conséquence** — plus aucun email ne doit partir vers l'adresse supprimée. Le protocole :

1. créer un deal → deux traces email écrites (`booking-requested-carrier` au Voyageur, `payment-authorized-shipper` à l'Expéditrice) ;
2. envoyer un rebond dur signé sur l'adresse du Voyageur → `suppressed: true` ;
3. créer un **second** deal identique.

Résultat : pour le second deal, l'événement `booking.requested` produit **zéro** trace email, tandis que `booking.payment_authorized` en produit une pour l'Expéditrice. Le résolveur de destinataires fait donc exactement ce que la règle exige — et la preuve ne repose pas sur la lecture du code.

Comment les fiches Stripe ont été jouées sans la CLI Stripe

La CLI n'était pas installée. Plutôt que de marquer trois fiches (dont deux **bloquantes**) en ►►, les événements ont été **signés à la main**, exactement comme Stripe les signe : HMAC-SHA256 de `<timestamp>.<corps brut>` avec le secret d'endpoint, en hexadécimal, dans l'en-tête `stripe-signature: t=...,v1=...`.

Un détail a coûté un premier échec, et mérite d'être noté : `$(cat fichier)` **retire le saut de ligne final**, alors que `curl --data-binary @fichier` l'envoie. La signature portait donc sur des octets différents de ceux transmis, et le service répondait 400 — **correctement**. La signature doit être calculée sur les **octets exacts du fichier**, lus en binaire. C'est la même exigence qui explique pourquoi cette route ne passe jamais par la passerelle : elle analyse puis re-séréalise le JSON, ce qui déplacerait ne serait-ce qu'une espace et casserait la signature. Vérifié au passage : `POST /api/webhooks/stripe` répond **404** — la route n'est pas exposée par la passerelle.

Rejeu avec la CLI Stripe — la réserve n° 1 est levée

La CLI a été installée (`brew install stripe/stripe-cli/stripe, 1.50.10`) et lancée **sans connexion interactive**, en lui passant directement la clé de test du dépôt :

```
stripe listen --api-key "$STRIPE_SECRET_KEY" --forward-to localhost:6003/webhooks/stripe
# Ready! ... Your webhook signing secret is whsec_...
```

Ce secret a été posé dans le `.env`, deal-service redémarré — **fournisseur Stripe réel cette fois**, plus le FAKE de la campagne.

API-HOOK-01 — OK. La CLI annonce le secret et journalise chaque événement avec le code rendu par le service.

API-HOOK-02 — OK, quatre réponses confirmées sur de vraies signatures.

Cas	Résultat
stripe trigger charge.succeeded (et les payment_intent.* de sa séquence)	200 — types non traités, ignorés volontairement
en-tête stripe-signature absent	400 Missing stripe-signature header.
signature invalide	400 Invalid webhook signature.
sans STRIPE_WEBHOOK_SECRET	501 (mesuré plus tôt dans la campagne)

API-HOOK-03 — OK, et cette fois de bout en bout sur une autorisation RÉELLE. C'est la vraie valeur ajoutée du rejeu : au lieu d'un événement fabriqué, on a suivi le chemin complet.

1. Devis puis intention de paiement par l'API → pi_3UDUp1... créé chez Stripe ;
2. stripe payment_intents confirm ... --payment-method pm_card_visa → **requires_capture**, amount_capturable: 1450, capture_method: manual — exactement le modèle de Yamba ;
3. POST /api/deals → **201**, deal PENDING sur cette autorisation réelle ;
4. stripe payment_intents cancel ... → **Stripe émet** payment_intent.canceled, la CLI le transmet signé, le service répond **200** ;
5. le deal passe **CANCELLED**, closedBy: SYSTEM, cancelReason: PAYMENT_AUTHORIZATION_LOST.

account.updated d'un compte connecté inconnu (stripe trigger account.updated, événement *connect*) → **200** et un avertissement au journal, sans plantage : le comportement voulu quand l'événement ne concerne aucun Voyageur de la base.

API-HOOK-04 — OK. stripe events resend deux fois sur le même evt_... : **200 aux trois livraisons**, statut du deal inchangé, et toujours un seul booking.cancelled dans la boîte d'envoi.

Un détail relevé au passage : la vue Expéditeur ne sert pas cancelReason (elle rend null), alors que la base porte bien PAYMENT_AUTHORIZATION_LOST. Ce n'est pas une fuite — c'est l'inverse : une information utile que le membre ne voit pas (« votre autorisation bancaire a expiré »). À arbitrer côté produit, pas un défaut de sécurité.

Axe non demandé — les deux webhooks n'avaient aucun test de contrôleur

Les règles pures étaient couvertes ; la **glu** ne l'était pas. Or c'est exactement la couche où la campagne a trouvé AN0-API-16 et AN0-API-17, et c'est ici que se décide une écriture lourde de conséquences : mettre une adresse sur liste de suppression coupe **tous** les emails d'un membre.

Deux specs ont donc été ajoutés dans cette PR :

- email-webhook.controller.spec.ts (8 cas) — 503 sans secret, 401 en-têtes absents, 401 mauvais secret, **401 horodatage hors tolérance** (anti-rejeu, non couvert jusqu'ici), 200 livraison, suppression **une seule fois**, rebond transitoire qui ne supprime pas, type ignoré ;
- stripe-webhook.controller.spec.ts (10 cas) — le tableau des quatre réponses, l'essai du second secret (Connect, A87), l'annulation, les drapeaux du Voyageur avec relance des versements, le compte inconnu qui ne fait pas planter, payout.failed, et le **500 sur échec transitoire** — celui-là est important : rendre 200 sur une base indisponible perdrait l'événement pour toujours.

notification-service 99 → **107**, deal-service 542 → **552**.

Chapitre 9 — Consignation et verdict de campagne

9.1 Ce que la campagne a couvert

146 fiches, 146 jouées. Plus aucune en ►► : API-H00K-01 (mise en place de l'écoute Stripe locale) a d'abord été contournée par des événements signés à la main, puis **rejoué le soir même avec la vraie CLI**, ainsi que les trois fiches Stripe qui en dépendaient.

Gravité	Jouées	OK du premier coup	KO → corrigé	Partiel	►►
Bloquante	54	43	11	0	0
Majeure	67	56	10	1	0
Mineure	25	21	2	2	0
Total	146	120	23	3	0

Vingt-deux anomalies, vingt-deux closes. Aucune n'a été « acceptée avec contournement » : toutes ont été corrigées, testées et contre-évaluées pendant la campagne.

9.2 Le tableau de suivi a lui-même été réparé

Une rebase avait laissé **seize lignes en double** dans le §9.1 du cahier (les blocs `API-MSG-*` et `API-NOTIF-*` figuraient deux fois, une version remplie et une version vide, dans un ordre mêlé). Le tableau annonçait 162 lignes pour 146 fiches. Il a été dédoublonné — en gardant systématiquement la version **remplie** — et remis dans l'ordre des chapitres.

C'est un défaut de consignation, pas de code, mais il méritait d'être noté : un tableau de suivi qui compte faux est exactement ce qui permet à une fiche de disparaître sans que personne s'en aperçoive.

9.3 Les quatre critères de sortie, un par un

1. Zéro anomalie bloquante ouverte — TENU

Les onze bloquantes trouvées sont fermées et contre-évaluées :

Anomalie	Ce qui était en jeu
ANO-API-01	un curseur invalide sortait en 500 non géré
ANO-API-02	404 et 400 étaient discriminables au temps de réponse
ANO-API-06	<code>/auth/me</code> renvoyait le secret TOTP du membre
ANO-API-08	« mot de passe oublié » trahissait l'existence du compte par sa durée
ANO-API-09	l'export RGPD répondait 500 — obligation légale inopérante
ANO-API-13	la page de suivi du destinataire répondait 500
ANO-API-16	une notification de messagerie rendait la boîte entière illisible
ANO-API-18	la connexion permettait d'énumérer les comptes par le temps (membre ET admin)
ANO-API-19	une course sur les derniers kilos rendait 500 au perdant
ANO-API-12 / ANO-API-15	valeur déclarée non confrontée aux plafonds ; destinataire trop exposé

2. Toutes les fiches bloquantes jouées et OK — TENU

Zéro bloquante en ►►. Deux d'entre elles ont failli y rester et ont été jouées en levant l'obstacle plutôt qu'en le contournant :

- API-SEC-05 (permission admin manquante) exigeait une **session administrateur avec TOTP**, ce que le cahier jugeait « hors de portée d'une campagne API ». Un profil SUPPORT a été posé sur un compte d'essai, la connexion en deux temps jouée, et le code TOTP **calculé** avec la bibliothèque du dépôt. Résultat : 403 sur `exports.personal`, `audit.read`, `finances.read`, la permission manquante étant nommée ; les routes du profil restent accessibles. Profil retiré après la fiche.
- API-SEC-12 / API-SEC-13 (signatures) attendaient le chapitre 8 : fermées là-bas.

API-AUTH-24 (export sans données d'autrui), qui était bloquée par ANO-API-09, a été **rejouée** après correction : export complet, aucune donnée d'un tiers.

3. Les majeures arbitrées une par une — TENU

Les dix majeures trouvées ont toutes été **corrigées**, aucune acceptée avec contournement. Reste une fiche majeure en écart : API-TRIP-18, la relance d'un lien de vérification n'est pas idempotente (inscrite au journal de dette ci-dessous).

4. Les mineures inscrites au journal de dette — TENU (ci-dessous)

9.4 Journal de dette au 8 septembre 2026

#	Constat	Fiche	Pourquoi ce n'est pas bloquant	Ce qu'il faudrait faire
~~D-1~~	~~x-locale ne pilote pas le formatage : c'est preferredLocale qui gagne~~	API-GW-10, API-MSG-11	—	SOLDÉE le 09/09 : l'arbitrage était déjà rendu au registre (D44) — le cahier avait tort. La vraie dette était ailleurs : trois endpoints écrivaient la règle de trois façons. Une seule fonction (<code>resolveViewerLocale</code>), appliquée partout, + garde-fou
D-2	trip-service rend parfois 400 là où 403 ou 404 seraient exacts	API-TRIP-13	écart de sémantique connu, sans fuite d'information	aligner sur la règle 403/404 du reste de la plateforme

#	Constat	Fiche	Pourquoi ce n'est pas bloquant	Ce qu'il faudrait faire
~~D-3~~	~~supprimer un document de trajet n'est pas idempotent : le rejeu répond 400~~ (l'intitulé du journal parlait à tort d'un « lien de vérification » — c'est la suppression d'un document)	API-TRIP-18	—	SOLDÉE le 09/09 : 200 « déjà supprimé » au rejeu, base supprimée avant le fichier, garde-fou <code>idempotent-delete.spec.ts</code>
~~D-4~~	~~les refus de trip-service et auth-service ne portent pas tous un <code>details.code</code> ~~	observé au chapitre 9	—	SOLDÉE le 08/09 au soir : 250 refus codés, garde-fou posé sur les deux services, et deux défauts de sémantique tombés avec (voir ci-dessous)
~~D-5~~	~~les middlewares écrivent leur réponse eux-mêmes , hors du middleware d'erreur~~	API-SEC-05	—	SOLDÉE le 08/09 au soir : douze refus passés par <code>next()</code> , le middleware d'erreur recopie <code>code</code> en tête pour ne casser aucun client, garde-fou <code>middleware-responses.spec.ts</code> . Deux exceptions écrites et justifiées (webhook Stripe, réponse documentée <code>ERASURE_BLOCKED</code>)

9.5 Ce que cette campagne NE prouve pas

Le cahier l'exige, et c'est la partie la plus honnête d'un rapport de recette.

- **Rien sur la charge ni la tenue en durée.** Le limiteur a été vérifié comme **garde métier** (100 / 1 000 requêtes), pas comme protection contre un déni de service distribué.
- **Rien sur les tâches planifiées** : expiration à 24 h, versement à 4 jours, révélation des avis à 14 jours, purges de rétention. Elles relèvent du cahier n° 4.
- **Rien sur la cohérence des événements** au-delà de leur effet observable par l'API. La boîte d'envoi, le relais, le dédoublonnage et les événements parqués relèvent du cahier n° 4.
- **Rien sur les écrans.** Une API conforme derrière un écran fautif reste un défaut.
- **Peu sur Stripe en conditions réelles** : la campagne a tourné sur le fournisseur **FAKE**, à l'exception du rejeu final des fiches webhook, joué sur le vrai Stripe en mode test (autorisation confirmée par carte d'essai, puis annulée chez le fournisseur). Restent non éprouvés : la capture, le versement au Voyageur, le remboursement et le renversement de transfert **en conditions réelles**.
- **Rien sur l'administration** au-delà des deux fiches jouées ici : le back-office a son propre cahier.

9.6 Verdict

La campagne est acceptée. Les quatre critères de sortie du §9.3 du cahier sont tenus : zéro bloquante ouverte, zéro bloquante non jouée, majeures toutes corrigées, mineures inscrites au journal de dette.

Plus aucune réserve. Les deux qui restaient au verdict ont été levées le soir même :

1. ~~rejouer les fiches Stripe avec la CLI~~ — fait (voir « Rejeu avec la CLI Stripe », chapitre 8) ;
2. ~~passer trip-service et auth-service au garde-fou `refusal-codes.spec.ts`~~ — fait (voir « Solde de la dette D-4 » ci-dessous).

~~Rejouer les fiches Stripe avec la CLI et un compte de test~~ — **fait le 08/09 au soir** : CLI installée, événements réels signés par Stripe, et surtout une **autorisation réelle annulée chez le fournisseur** qui annule bien le deal côté Yamba. Voir « Rejeu avec la CLI Stripe » au chapitre 8.

9.7 Ce que la campagne a appris, au-delà des anomalies

Un test qui ne trouve rien doit être suspecté avant d'être cru. Le balayage du code de livraison a été écrit trois fois ; les deux premières versions annonçaient « code absent partout » et ne testaient rien. Ce qui a sauvé la troisième : un **témoin positif** — exiger que le code soit *présent* là où il est légitime.

Corriger là où le défaut a été vu ne corrige pas le défaut. AN0-API-19 avait été refermée sur la création de deal ; la même cause est ressortie deux fiches plus loin (AN0-API-21). Le bon geste était de remonter au writer commun. Même histoire pour AN0-API-09 → AN0-API-13, puis AN0-API-20 → AN0-API-04.

Une règle qu'un outil rend impossible n'est pas une règle. « Tout refus métier porte un `details.code` » était écrit noir sur blanc — et inapplicable pour les 403 et 404, dont les classes d'erreur n'acceptaient pas de `details`. Aucune relecture n'aurait attrapé cela.

Un mock ne dit jamais ce qu'on lui invente. Deux 500 en production potentielle (AN0-API-09, AN0-API-13) venaient de champs Prisma inexistant, dans du code **couvert par des tests** qui injectaient un faux client. La réponse a été une famille de tests qui **lisent les sources** et les confrontent au schéma — sept au total aujourd'hui.

Enfin : la moitié des anomalies de cette campagne ne sont pas des fautes de logique, mais des refus mal formulés. Le serveur savait ce qu'il faisait ; il ne savait pas le *dire*.

Solde de la dette D-4 — « tout refus métier porte un `details.code` », partout

Le verdict de campagne inscrivait au journal de dette : la règle était tenue sur `deal-service` (ANO-API-04) et `message-service` (ANO-API-20), pas sur **trip-service** ni **auth-service**. Elle l'est maintenant sur les quatre, et sur les middlewares partagés.

Ce qu'il y avait à faire

Service	Refus métier sans code	Erreurs de forme (exclues)
trip-service	81	8
auth-service	169	26

Les erreurs de forme (schéma Zod) restent hors périmètre : elles portent déjà la liste des champs fautifs, **qui est leur contrat** — le client y lit quel champ corriger.

Deux défauts de sémantique sont tombés avec la dette

C'est le vrai gain, et il n'était pas dans l'énoncé. En codant les refus de `trip-service`, la même fonction est apparue neuf fois :

```
const { trip, error } = await findOwnedTrip(id, userId);
if (!trip) return next(new ValidationError(error)); // ← 400, toujours
```

`findOwnedTrip` renvoie « Trip not found. » ou « Unauthorized. » — deux situations que la règle non négociable de la plateforme distingue par le statut (**404** et **403**), et que ce code écrasait en un seul **400**, sans code. Un 400 dit « votre saisie est mauvaise » : ni l'une ni l'autre ne l'était.

C'était la dette **D-2** du même journal (« `trip-service` rend parfois 400 là où 403 ou 404 seraient exacts »), sur ces neuf sites. Corrigée en même temps :

```
const { trip, error, code } = await findOwnedTrip(id, userId);
if (!trip) return next(ownershipError(code, error)); // 404 TRIP_NOT_FOUND | 403 NOT_TRIP_OWNER
```

Vérification faite avant de changer les statuts : **aucun écran du front ne branche sur le 400** de ces routes (les seuls `status === 40x` du front portent sur la notation, le suivi de deal, la messagerie et les profils publics).

Les middlewares partagés — la dette D-5, à moitié soldée

`isAuthenticated` écrit ses réponses lui-même, sans passer par le middleware d'erreur. Trois de ses sept refus portaient un `code`, **les quatre autres rien du tout** — dont « jeton absent », le 401 le plus fréquent de la plateforme. Les sept ont désormais un code distinct :

Refus	Code	Ce que le client doit en faire
jeton absent	<code>TOKEN_MISSING</code>	se connecter
jeton illisible	<code>TOKEN_INVALID</code>	se connecter
jeton expiré ou invalide	<code>TOKEN_EXPIRED</code>	rafraîchir, puis se connecter
compte introuvable	<code>USER_NOT_FOUND</code>	se connecter
compte effacé	<code>ACCOUNT_DELETED</code>	ne pas réessayer
compte suspendu	<code>ACCOUNT_SUSPENDED</code>	contacter le support
session révoquée	<code>SESSION_REVOKED</code>	se reconnecter (on l'a déconnecté ailleurs)

`authorizeRoles` a reçu `ROLE_NOT_ALLOWED`. Ce qu'il reste de D-5 — faire passer ces middlewares par le middleware d'erreur commun, pour n'avoir qu'une seule forme de corps d'erreur — est un chantier de forme, sans effet fonctionnel : il reste au journal.

Les garde-fous

`apps/trip-service/src/lib/refusal-codes.spec.ts` et `apps/auth-service/src/utls/refusal-codes.spec.ts` — mêmes que ceux de `deal-service` et `message-service` : ils lisent les sources et refusent tout jet d'erreur métier sans code.

Un détail a demandé une deuxième passe : un code peut s'écrire de **trois** façons légitimes — `{ code: "X" }`, le raccourci `{ code }` quand la variable porte déjà le nom, et un objet `details` construit ailleurs et passé tel quel. La première version du test ne reconnaissait que la première et criait au loup sur trois sites corrects. Un garde-fou trop littéral fabrique du faux positif, et le faux positif est ce qui fait désactiver les garde-fous.

Le test d'`auth-service` vérifie en plus que trois codes de 401 **différents** existent bien (`INVALID_CREDENTIALS`, `SESSION_EXPIRED`, `ACCOUNT_SUSPENDED`) : c'est tout l'intérêt de la règle, un client doit réagir différemment à chacun.

Contre-épreuves

Appel	Avant	Après
modifier un trajet inexistant	400 « Trip not found. »	404 <code>{"code": "TRIP_NOT_FOUND"}</code>
modifier le trajet d'un autre	400 « Unauthorized. »	403 <code>{"code": "NOT_TRIP_OWNER"}</code>
mettre en favori un trajet inexistant	400	404 <code>{"code": "TRIP_NOT_FOUND"}</code>
appel sans jeton	401 sans code	401 <code>{"code": "TOKEN_MISSING"}</code>
jeton forgé	401 sans code	401 <code>{"code": "TOKEN_EXPIRED"}</code>
mauvais mot de passe	401 sans code	401 <code>{"code": "INVALID_CREDENTIALS"}</code>
se suivre soi-même	400 sans code	400 <code>{"code": "CANNOT_FOLLOW_SELF"}</code>

`trip-service` 231 → **235**, `auth-service` 215 → **219**. Plateforme **941**.

Un piège Nx, payé une deuxième fois

La contre-épreuve a d'abord montré les **anciens** corps d'erreur alors que le bundle contenait bien le nouveau code. Cause : `npm run dev` lance `nx serve`, qui construit la cible `build:development` — et `nx build <service> --skip-nx-cache` ne réchauffe que `build`. Le serveur repartait donc sur un artefact en cache.

C'est la même leçon que le typecheck servi depuis le cache pendant la campagne : **un artefact Nx « reconstruit » n'est pas forcément neuf**. Pour redémarrer sur du code frais : `NX_SKIP_NX_CACHE=true npm run dev`.

Solde de la dette D-5 — une seule forme de corps d'erreur

Le journal de dette disait : « les middlewares écrivent leur réponse **eux-mêmes**, hors du middleware d'erreur ». Conséquence : **trois** formes de corps coexistaient sur la plateforme, et aucun client ne pouvait écrire UNE fonction pour les lire.

Forme	Qui la produisait
<code>{ status: "error", message, details }</code>	le middleware d'erreur commun
<code>{ message, code }</code>	<code>isAuthenticated</code> , <code>requireAdminPermission</code> , <code>requireActiveAccount</code>
<code>{ success: false, message }</code>	les 404 des pages publiques (profil, trajet)

Court-circuiter le middleware d'erreur coûte plus qu'une incohérence de forme : **aucune décision centrale** sur ce qui est exposé en production, et **aucune remontée Sentry**.

Ce qui a été fait

Le **middleware d'erreur recopie code au premier niveau** quand `details.code` existe. C'est ce qui rend la migration possible sans casser personne : les clients qui lisaient le code en tête continuent de le trouver, ceux qui suivent la règle générale lisent `details.code`. Sans cette recopie, solder D-5 revenait à casser des écrans.

Puis douze refus sont passés par `next()` : les sept de `isAuthenticated`, les quatre de `isAdminAuthenticated` (qui n'avaient **aucun** code), `requireActiveAccount`, `requireAdminPermission`, les cinq 404 `{success: false}` des pages publiques, le 409 Stripe du Voyageur et un 400 de session admin.

Ce qui reste volontairement en dehors, et pourquoi : les réponses du webhook Stripe (`{received: true}` / `{error}`) — leur destinataire est Stripe, pas un client Yamba, et leur contrat est le **statut**, pas le corps ; et le 409 ERASURE_BLOCKED, qui n'est pas un corps d'erreur ad hoc mais une **réponse documentée** avec son propre schéma OpenAPI. Les transformer changerait un contrat publié pour un gain de forme. C'est écrit ici pour que le choix soit relisible, pas oublié.

Garde-fou : `middleware-responses.spec.ts` interdit tout `res.status(4xx|5xx).json(...)` dans `packages/middleware` et exige un code sur chaque refus.

ANO-API-23 (bloquante) — les deux pages publiques répondaient 404, et un test protégeait le défaut

Trouvée **en soldant D-5**, en vérifiant une réponse qui semblait anodine.

```
ANO-API-23
Fiche      : hors cahier — trouvée au solde de la dette D-5 · Gravité : BLOQUANTE · ÉTAT : CLOSE
Appel exact : curl -s -o /dev/null -w '%{http_code}' "$BASE/users/madi-d-cmqhd/public"
            curl -s -o /dev/null -w '%{http_code}' "$BASE/trips/69e23ec5c5c73a77d8fed8e0/public"
Attendu    : 200 — le compte est public, le trajet est PUBLISHED
Obtenu     : 404 sur les deux. Mesuré sur toute la base :
            · profil public → 404 pour 22 comptes sur 26
            · page du trajet → 404 pour 24 trajets publiés sur 37
Cause      : `publicProfileWhere` filtre sur `isDeleted: { not: true }` et
            `profilePublic: { not: false }`, `publicTripWhere` sur `isDeleted: { not: true }`.
            Or ces champs sont ABSENTS des documents créés avant leur ajout au schéma — et
            sur Prisma + Mongo **aucun filtre ne matche un champ absent**, `not` compris.
            Prisma RELIT pourtant la valeur par défaut : le compte s'affiche `profilePublic:
            true` à la lecture et reste introuvable à la requête.
Impact     : la vitrine de la plateforme. Le profil public d'un Voyageur et la page d'un
            trajet — les deux seules pages qu'on partage à l'extérieur — invisibles pour la
            grande majorité des comptes et des trajets hérités.
Correction : (1) données — `repair-absent-scalars.ts` pose le défaut du schéma là où le champ
            manque (90 champs sur les comptes, 24 sur les trajets), idempotent, avec un
            `--dry-run` ; (2) code — les filtres reviennent à l'ÉGALITÉ SIMPLE, qui est la
            bonne écriture une fois les données saines.
Contre-épreuve : profil 200, les deux trajets hérités 200, et un trajet inexistant toujours
            404 `TRIP_NOT_FOUND`. Vérifié CONTRE LA BASE, pas seulement sur la forme.
```

Deux choses que cette anomalie apprend

Un commentaire n'est pas une vérification. Le fichier fautif affirmait, noir sur blanc : « `isDeleted: { not: true }` et `profilePublic: { not: false }` plutôt qu'une égalité, **pour matcher aussi les documents où le champ est ABSENT** ». La phrase est confiante, elle cite le bon piège maison — et elle est fausse. Elle a été écrite en corrigeant ANO-API-02, jamais éprouvée contre la base, et elle a survécu à la campagne entière.

Un test peut protéger le défaut au lieu de le trouver. Les deux specs existants exigeaient littéralement l'erreur :

```
it("n'utilise jamais `isDeleted: false`, qui raterait les documents sans le champ", () => {
  expect(JSON.stringify(publicTripWhere(ID))).not.toContain('"isDeleted":false');
});
```

Le test passait, la fonctionnalité était morte. Un test qui vérifie la **forme** d'une requête ne vérifie pas qu'elle **trouve** quelque chose : il fige la croyance de son auteur. C'est le pendant exact de la leçon d'ANO-API-09 (« un mock ne dit jamais ce qu'on lui invente »), un cran plus haut.

Et pourquoi la campagne ne l'avait pas vue

Le jeu d'essai **écrit** ces champs. Toutes les fiches jouées sur des données de seed passaient. C'est mot pour mot le piège déjà inscrit dans CLAUDE.md — « une fixture qui pose le champ ne prouve rien sur le vrai writer » — payé ici pour la sixième fois, et pour la première fois sur User et Trip.

Le remède complet, et sa limite

`isSet: false` est le réflexe du projet pour « champ absent ». **Il ne s'applique pas ici** : Prisma ne l'offre que sur les champs **optionnels**. Sur un champ requis à défaut, l'écrire lève `Unknown argument 'isSet' – vérifié contre la base (et vérifié aussi qu'il fonctionne bien sur hiddenByAdminAt, qui est DateTime?)`.

Donc, pour un champ REQUIS, « absent » n'est pas exprimable dans une requête : c'est un défaut de **données**, et le remède est le script de réparation, à passer après tout ajout d'un champ requis à défaut. Les documents créés ensuite sont sains d'office : Prisma écrit le défaut à la création.

Cette limite a failli me coûter une deuxième erreur du même genre : la première version du correctif posait un `OR ... isSet: false` sur les deux champs, les tests unitaires de forme passaient au vert, et le service répondait **500** au premier appel réel. Le garde-fou qui a servi n'est pas un test : c'est d'avoir rejoué la requête contre la base avant de conclure.

Solde de la dette D-3 — une suppression rejouée n'est pas une erreur

Ce que le journal disait, et ce qu'il fallait lire

L'entrée D-3 était **mal écrite** : elle parlait d'un « lien de vérification » alors que la fiche `API-TRIP-18` porte sur la **suppression d'un document de trajet**. L'intitulé est corrigé dans le tableau ci-dessus. Une dette mal nommée est une dette qu'on ne retrouve pas.

Le défaut

```
DELETE /api/trips/{id}/documents/{documentId} → 200 « Document removed. »
DELETE /api/trips/{id}/documents/{documentId} → 400 « Document not found. » ← le rejeu
```

Deux défauts dans un seul refus :

1. **Le geste n'était pas idempotent**. Un second clic, un rejeu réseau, un retour arrière du navigateur : le Voyageur voyait une erreur pour un geste qui avait parfaitement fonctionné.
2. **Le statut était faux**. 400 annonce une faute du client ; il n'y en avait aucune.

Et la route **voisine** — `DELETE /uploads/imagekit/:fileId` — appliquait déjà la bonne convention (« File was already deleted. » en 200). Deux suppressions côte à côte, deux comportements.

La correction

```
// Un document absent, ou appartenant à un AUTRE trajet, est traité comme déjà supprimé.
if (!doc || doc.tripId !== id) {
  return res.status(200).json({ success: true, message: "Document was already removed." });
}
```

Confondre « n'a jamais existé » et « déjà supprimé » n'est pas un pis-aller : c'est ce qui **ferme la porte à l'énumération d'identifiants**. L'appelant ne peut rien déduire, et rien n'est touché.

L'ordre des effets a été inversé au passage. L'ancien code supprimait le fichier chez ImageKit **puis** la ligne en base : si la seconde écriture échouait, il restait une ligne pointant vers un fichier disparu. Désormais la base — source de vérité — part d'abord ; au pire il reste un fichier orphelin chez le fournisseur, sans conséquence. L'échec du fournisseur reste absorbé : il ne fait pas échouer la suppression.

Trois statuts faux corrigés dans la même passe

La dette D-4 avait donné un `details.code` à tous les refus, mais **sans toucher aux statuts** : plusieurs `ValidationError` portaient un code d'authentification tout en répondant **400**.

Cas	Avant	Après
garde défensive <code>!req.user</code> (16 sites)	400 UNAUTHENTICATED	401 UNAUTHENTICATED

Cas	Avant	Après
alerte de trajet appartenant à un autre membre (3 sites)	400 UNAUTHENTICATED	403 NOT_OWNER

Le second cas était le plus trompeur : « UNAUTHENTICATED » sur un membre parfaitement authentifié, à qui il manquait seulement la propriété de la ressource.

Garde-fou

apps/trip-service/src/lib/idempotent-delete.spec.ts (5 cas) : il lit les sources et vérifie que la suppression d'un document ne rend aucun refus sur l'absence, que les deux cas d'absence partagent **la même sortie** (pas d'énumération), que la base est supprimée **avant** le fichier, que l'échec du fournisseur est absorbé, et que la route voisine garde sa convention.

Contre-épreuves

Appel	Résultat
supprimer un document, trois fois de suite	200 · 200 « déjà supprimé » · 200
supprimer le document d'un autre trajet	200 « déjà supprimé », rien touché
supprimer sur le trajet d'autrui	403 NOT_TRIP_OWNER (inchangé)
supprimer une alerte de trajet inexistante	200 (elle était déjà idempotente)
appeler une route d'upload sans session	401 TOKEN_MISSING

trip-service 235 → **240**. Plateforme **946**.

Le journal de dette ne contient plus qu'une ligne : D-1, l'arbitrage produit entre l'en-tête `x-locale` et le `preferredLocale` du membre — qui n'est pas un défaut, mais un choix à trancher.

Solde de la dette D-1 — une seule règle pour la langue d'une réponse

L'arbitrage n'était pas à rendre : il était déjà au registre

Le journal présentait D-1 comme un choix à trancher — « soit l'en-tête prime, soit le cahier est corrigé ». En allant lire, la décision **D44** dit exactement ceci :

③ Flux **avec compte** : `preferredLocale`, initialisée à l'inscription depuis `x-locale` et **mise à jour immédiatement quand un utilisateur connecté bascule la langue** [...] Une locale **par utilisateur (pas par appareil)**.

Et la seconde moitié est bien implémentée : `HeaderLocaleSwitcher` appelle `PATCH /auth/me/locale` à chaque bascule. Le code et le registre étaient d'accord ; **c'est l'attendu du cahier de recette qui était faux**, et la règle de précedence du projet le dit — code + tests > registre > synthèses.

Le cahier est corrigé (fiche API-GW-10 et les deux lignes de suivi).

La vraie dette était ailleurs, et le code la portait

En vérifiant, trois endpoints résolvaient la langue de **trois façons différentes** :

Endpoint	Règle appliquée	Ce qui manquait
GET /messages/quick-replies	compte → appareil	rien (la règle juste)
GET /trips/favorites	?locale → appareil	le compte n'était jamais consulté
GET /trips/search	?locale seul, défaut fr	ni compte, ni appareil

Aucune n'était fausse isolément. Ensemble, elles n'étaient pas la même règle — et la recherche répondait **toujours en français** à qui ne passait pas `?locale=`. Le front s'en sortait parce qu'il passe le paramètre ; **tout autre client de l'API recevait du français**, quel que soit le lecteur.

La règle, écrite une fois

`resolveViewerLocale` (dans `@packages/api-contracts`, là où vit déjà la liste des langues) :

1. `?locale= explicite` — l'appelant a demandé cette langue pour cet appel. C'est ce qui rend un lien partageable et une réponse d'API reproductible : une préférence stockée ne doit pas changer le résultat d'une URL qu'on s'est envoyée.
2. `preferredLocale` — D44, la langue du compte, pas de l'appareil.
3. `x-locale` — l'appareil, pour un visiteur sans compte.
4. `Accept-Language`, puis le défaut.

Un détail qui compte : une valeur **non supportée** ne consomme pas son tour. `?locale=de` sur un membre anglophone rend de l'anglais, pas le français par défaut — sinon un paramètre erroné écraserait silencieusement la préférence du lecteur.

Garde-fous

- `viewer-locale.spec.ts` (6 cas) — la règle elle-même, y compris le piège de la valeur inconnue ;
- `one-locale-rule.spec.ts` (4 cas) — **aucun contrôleur ne lit `x-locale` sans passer par la règle commune**. C'est ce test qui empêche l'apparition d'une quatrième variante.

Contre-épreuves sur le service

Appel	Résultat
réponses rapides, compte FR, appareil EN	« Je suis en route. » — le compte gagne
après <code>PATCH /auth/me/locale</code> en <code>en</code> , appareil FR	« I'm on my way. » — la bascule est immédiate
recherche, visiteur anonyme, appareil EN / FR	September 11, 2026 / 11 septembre 2026
recherche, membre EN , appareil FR, sans paramètre	September 11, 2026 — avant : toujours du français
recherche, membre EN, <code>?locale=fr</code>	11 septembre 2026 — la surcharge explicite gagne

trip-service 240 → **250**. Plateforme **956**.

Le journal de dette est vide. Les cinq lignes ouvertes à la fin de la campagne (D-1 à D-5) sont soldées, et les vingt-trois anomalies closes.